

课程笔记

Lecture 1. Introduction and Basics - Carnegie Mellon - Computer Architecture 2015 - Onur Mutlu

Carnegie Mellon Computer Architecture 2015 / Onur Mutlu

Codex 中文整理版



视频频道: Carnegie Mellon Computer Architecture

发布日期: 2015-01-21

视频时长: 01:54:35

视频链接: https://www.youtube.com/watch?v=zLP_X4wyHbY&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq

目录

1	建筑式设计心智：从 Fallingwater 开始	3
1.1	为什么计算机体系结构课从一栋房子开始	3
1.2	第一次作业：用差异训练设计观察力	3
1.3	Wright 为什么能设计出 Fallingwater	5
1.4	原则化设计：organic architecture 给出的类比	5
1.5	本章小结	6
2	计算机体系结构师必须预测并塑造什么	6
2.1	架构师的四个方向：向后、向前、向上、向下	6
2.2	向上看：从 problem domain 出发	7
2.3	向下看：技术是材料，也是边界	8
2.4	预测与适应：平台寿命把今天的选择推向未来	8
2.5	工程分析与判断力要一起工作	9
2.6	为什么 architectural innovation 能改变日常系统	10
2.7	本章小结	11
3	作为硬件-软件接口的体系结构	11
3.1	为什么要从“中间层”看计算机	11
3.2	程序员视图、硬件设计视图与架构师视图	12
3.3	多核系统：硬件选择怎样改变软件成本	12
3.4	实验中的硬件与课程真正关注的边界	13
3.5	本章小结	14
4	转换层级与抽象的力量	14
4.1	计算的目的：从问题到洞见	14
4.2	从问题到电子：接口怎样组织复杂性	15
4.3	Hamming distance：一个形式化接口的微型例子	16
4.4	Hamming 的研究建议：想法需要被清楚表达	17
4.5	抽象带来的生产力	17
4.6	抽象的代价：可见性不足会阻碍优化	18
4.7	本段课程目标：理解平台行为，做跨层决策	19
4.8	本章小结	19
5	案例一：多核减速、DRAM bank 与调度公平性	20
5.1	从一个反直觉的减速现象开始	20
5.2	为什么 OS priority 没有修好它	22
5.3	DRAM bank、row 与 row buffer	23
5.4	FR-FCFS 风格调度：局部高效如何放大成不公平	24
5.5	为什么一个 row 可以连续压住 128 个 cache block	25
5.6	从拒绝服务到公平 memory controller	26
5.7	本章小结	28
6	案例二：DRAM Refresh、扩展压力与 Retention-Aware Design	28
6.1	从共享资源干扰走向器件物理	28
6.2	64 ms refresh window 与刷新次数	29
6.3	刷新开销怎样限制 DRAM scaling	30
6.4	Retention variation：为什么统一刷新会浪费	31
6.5	Retention-aware refresh：按 row 的真实需要调度	32

6.6	Bloom filter (布隆过滤器): 用紧凑 metadata 表示弱 row 集合	34
6.7	收益、边界与跨层含义	36
6.8	本章小结	37
7	案例三: RowHammer 与缩放下的可靠性	37
7.1	从 refresh 走向 disturbance: 第三个 DRAM 案例的动机	37
7.2	基本模型: 攻击行、受害行与电荷阈值	38
7.3	物理机制: 反复 activate/precharge 怎样影响相邻 row	39
7.4	为什么正常 refresh 可能来不及	40
7.5	商用芯片观察: 隐藏电路效应进入真实系统	41
7.6	跨层影响: 可靠性问题怎样变成安全和系统问题	42
7.7	缓解思路: 从器件改良到相邻行刷新	42
7.8	更宽的设计空间: 可靠内存与可容错内存	43
7.9	本章小结	44
8	课程机制与如何使用这堂课	44
8.1	把课程网站当作统一入口	44
8.2	作业、实验与 recitation 的配合	45
8.3	阅读、写作与时间管理	46
8.4	本章小结	46
9	总结与延伸	46
9.1	一条主线: 有原则的跨层设计	46
9.2	从共享资源到 scaling: 两个案例的共同结构	47
9.3	课程方法: 用刻意练习打通抽象	47
9.4	可带走的判断、开放问题与下一步	48

1 建筑式设计心智：从 Fallingwater 开始

1.1 为什么计算机体系结构课从一栋房子开始

00:00:40 左右, 讲者暂缓处理器、缓存和指令集的定义, 先问学生屏幕上的建筑是什么。学生认出 Fallingwater (流水别墅) 之后, 讲者继续追问设计者是谁, 并把答案指向 Frank Lloyd Wright。这个开场的教学意图很明确: 先让学生进入“设计”这个更大的问题, 再把这种设计心智迁移到计算机体系结构。

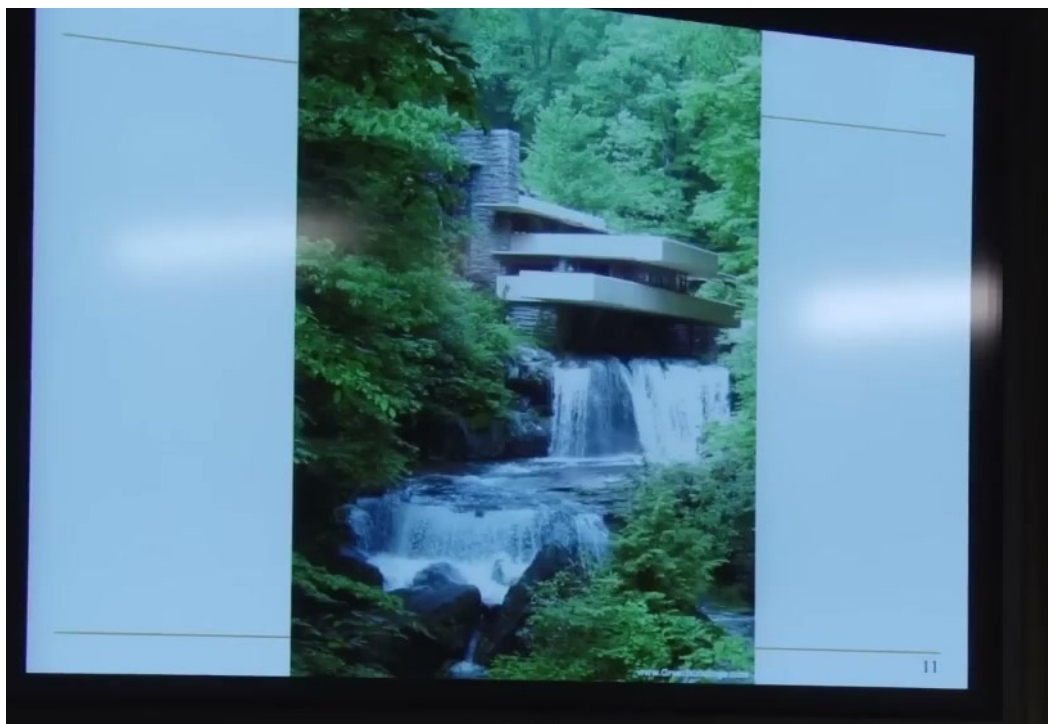


图 1: Fallingwater 把建筑放入瀑布与森林环境中, 展示了设计如何同时处理场地、结构和审美约束。¹

Fallingwater 的事实背景在 00:01:00–00:02:04 被简要交代: 它离 Pittsburgh 很近, 由 Frank Lloyd Wright 设计, 1935 年完成, 并被作为值得参观的重要建筑来介绍。这些信息提供了一个可观察的设计对象: 一栋建筑怎样处理地形、水流、结构、居住体验、成本和审美之间的关系。

Fallingwater 在这里承担的教学角色

Fallingwater 是一个“约束中的设计”例子。建筑物要服务人的居住目标, 要承受材料和结构限制, 要回应场地环境, 还要体现设计者的审美与原则。计算机系统也类似: 它要服务 workload, 要受制于器件、能耗、成本、可靠性和软件接口, 还要在未来几年继续可用。这个类比使读者先看到设计问题的全貌, 再进入具体硬件机制。

因此, 本节开头的核心问题可以这样表述: 体系结构课程训练两层能力: 先识别硬件部件, 再解释组织方式、目标和代价。

1.2 第一次作业: 用差异训练设计观察力

00:02:16–00:03:44, 讲者把参观 Fallingwater 说成这门课的“first assignment”。他给出的理由包括: 学生可以欣赏 out-of-the-box 和 creative thinking 的重要性, 可以思考建筑设计中的 strengths、

¹视频画面时间区间: 00:08:26–00:09:31。若图像为裁剪图, 时间区间仍对应原视频画面。

weaknesses 和 tradeoffs, 也可以在学完课程后推导出自己的 good design and innovation principles。

这段话把“作业”的重点放在观察和解释上。学生需要做的事情类似工程师面对两个候选系统时的分析：先找出差异，再问这些差异来自什么目标和约束。比如一个处理器设计可能增加缓存来降低访存延迟，也可能选择更简单的缓存组织来控制面积和功耗。单看某个局部选择，很容易把“更复杂”误认为“更好”；放回目标和约束中，评价才有意义。

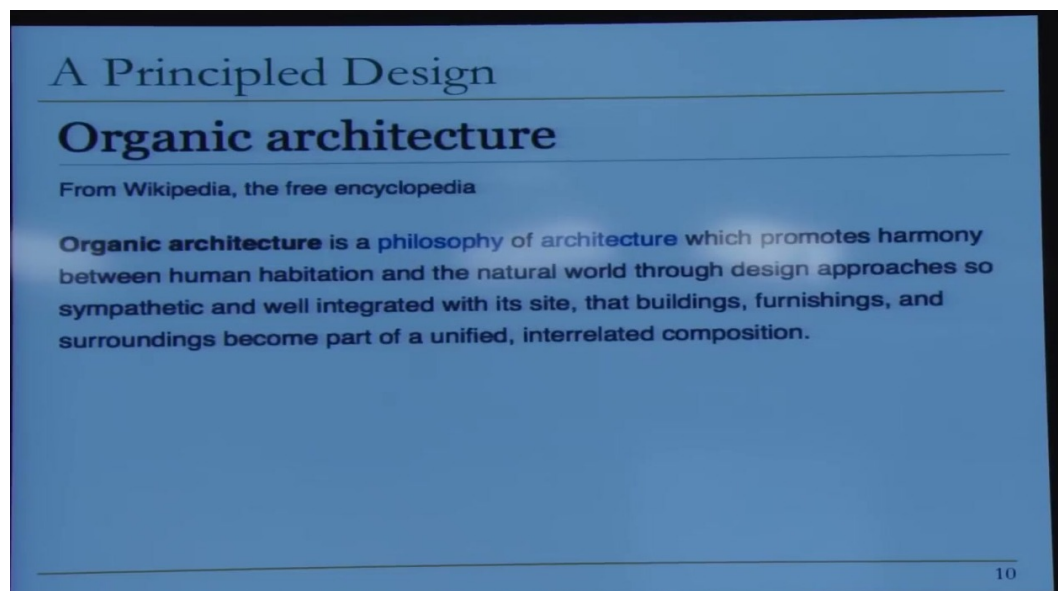


图 2: 有机建筑的定义说明，原则化设计追求建筑、使用者与环境之间的统一关系。²

00:03:44–00:05:48，讲者展示两栋建筑，让学生找 differences。他指出其中一栋更 traditional，另一栋更 impressive、more creative，并且更 in line with the surroundings。随后他马上引入 cost：Fallingwater 更昂贵，甚至昂贵到委托方 Kaufmann 要求 Wright 停止继续扩展设想。这里的教学点很硬：设计没有免费午餐，创造性常常带来成本、施工难度、维护负担或其他代价。

好设计的评价必须带着目标和约束

00:05:27–00:05:48 的关键句是：a good design is dependent on what it is designed for。换成体系结构语言，一个设计 D 的好坏要放在目标 G 、约束 C 、环境 E 中评价：

$$\text{value}(D) = F(D \mid G, C, E)$$

- D : 具体设计，例如某种处理器、缓存层次、内存控制策略，或这里的建筑方案。
- G : 设计目标，例如居住体验、性能、能效、可靠性、成本或可编程性。
- C : 约束，例如预算、材料、工艺、功耗、面积、时间。
- E : 环境，例如 Fallingwater 的瀑布与森林，或计算机系统面对的软件生态、工作负载和技术趋势。
- F : 评价函数，表示目标、约束和环境共同决定设计价值。

这个公式是对讲者 00:04:45–00:05:48 设计取舍讨论的压缩表达，用来提醒读者：脱离使用目的的“最佳设计”说法很容易误导。

²视频画面时间区间：00:07:52–00:08:26。若图像为裁剪图，时间区间仍对应原视频画面。

常见误解：把差异看成优劣清单

讲者让学生找两栋建筑的 differences, 并没有要求学生把其中一栋简单判为优胜者。00:04:45–00:05:26 已经给出成本例子：一个方案可能更有创造性、更贴合环境，同时也更昂贵、更难实现。体系结构中的缓存、流水线、乱序执行、内存控制器也有同样性质：每个机制都要问收益落在哪里，代价由谁承担。

1.3 Wright 为什么能设计出 Fallingwater

00:05:48–00:07:28, 讲者把问题推进到 “what it means to be a good architect”。学生先提出 creativity, 讲者接着列出 hard work、perseverance、dedication over decades、experience、out-of-the-box thinking、principled design、understanding of past designs、good judgment and intuition, 以及 architecture、math、art 等多种 skill 的组合。

这组词在课程里承担架构师能力模型的作用，重点落在可训练、可积累、可检验的设计能力上。creative thinking 让设计者敢于提出新组织方式；experience 和 past designs 让设计者知道哪些路径曾经有效、哪些路径曾经失败；technical skills 保证想法能落到可构造系统；judgment and intuition 让设计者在信息不完整时做出合理选择。计算机体系结构会反复遇到这种情况：未来 workload 未知，器件趋势未完全确定，成本和可靠性约束也在变化，设计者仍然必须提交一个可执行方案。

precedent 与 principle 的关系

precedent 指已有设计、已有技术路线和已有失败经验；principle 指从这些经验中抽象出的可迁移规则。只看 precedent 容易复制旧方案，只喊 principle 容易脱离工程细节。讲者在 00:09:35–00:10:24 强调要理解 principles, 也要理解 precedents, 因为理解过去能帮助学生发展自己的原则，并评估不同设计和 idea 的 tradeoffs。

1.4 原则化设计：organic architecture 给出的类比

00:07:28–00:08:33, 讲者引用 Wright 的设计观，并介绍 organic architecture：建筑、家具和周围环境应形成统一而相互关联的组合。00:08:42–00:09:31, 他用 Fallingwater 的具体形态解释这个原则：建筑放在瀑布之上，本身难以建造；它的水平伸出结构呼应上下两段瀑布，同时需要足够的 technical insight 和 technical aptitude 才能稳固地停留在瀑布上方。

这正是本节从建筑转向计算机体系结构的桥梁。体系结构里的“有机”指组件、约束和目标之间的协调关系。例如，一个高性能内存系统需要降低单个请求延迟，也需要协调 row buffer locality、多个核心的公平性、能耗、容量、可靠性和软件访问模式。后续课程会展开这些机制，本节先建立判断框架：原则要能解释结构，结构要能承受约束。

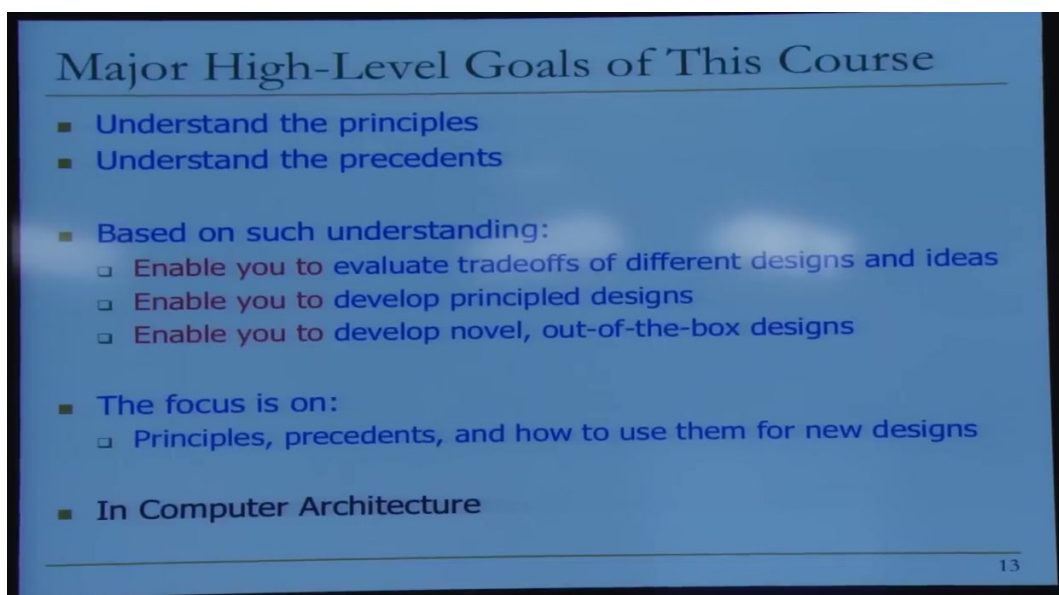


图 3: 课程目标强调原则、先例、取舍和新设计能力, 把建筑类比转回计算机体系结构。³

本课程对“原则”的使用方式

00:09:35–00:10:41, 讲者把课程目标说成三件事: 理解设计原则, 理解历史 precedent, 学习能够用于新设计的技术。技术是工具, principle 给出判断方向, precedent 提供经验边界。学生掌握三者之后, 才可能评价 design tradeoffs, 并提出 novel out-of-the-box designs。

建筑类比的边界

讲者在 00:10:41–00:11:11 明确提醒学生, 这门课仍然是 computer architecture。Fallingwater 的作用是训练设计观察力和原则化思维; 后续讨论会转入 ISA、microarchitecture、memory system、workload、technology trend 等计算系统对象。类比能打开视角, 实际论证仍要回到可测量的系统机制。

1.5 本章小结

本节用 00:00:40–00:11:11 的 Fallingwater 开场建立了一个设计判断框架。第一, 优秀设计要放进目标、约束和环境中评价, 成本、创造性、适配性和可建造性共同决定结果。第二, 架构师需要 creativity、experience、technical skills、judgment 和 intuition 的组合。第三, principles 与 precedents 共同构成课程方法: 理解过去的设计经验, 抽象出可迁移原则, 再用技术机制实现新的系统方案。这个框架会在后续计算机体系结构内容中不断出现。

2 计算机体系结构师必须预测并塑造什么

2.1 架构师的四个方向: 向后、向前、向上、向下

00:11:11–00:13:52, 讲者从 Fallingwater 类比进入 computer architect 的角色。下文将 computer architect 译作“体系结构师”, 简称“架构师”。他先借用导师提出的架构师职责, 再按自己的方式扩展为四个方向: look backward、look forward、look up、look down。这四个方向把架构师放在时间轴和系统层次的交叉点上。

³视频画面时间区间: 00:09:35–00:10:43。若图像为裁剪图, 时间区间仍对应原视频画面。

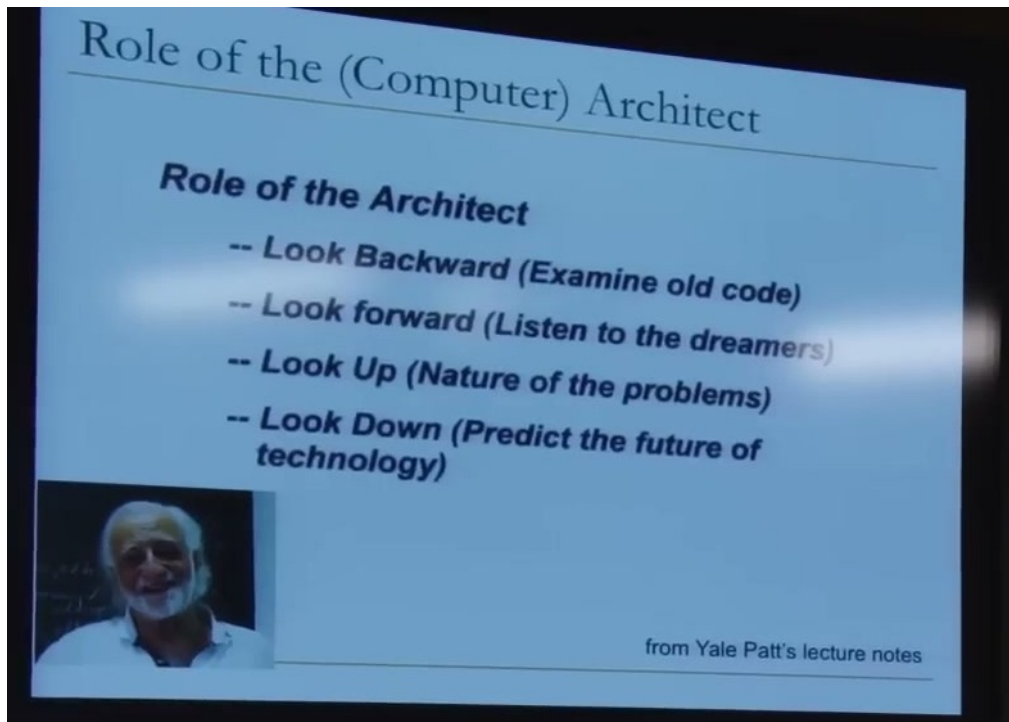


图 4: 架构师需要回看先例、前看趋势、上看应用、下看技术，把多个方向的信息合成设计判断。⁴

computer architect 的四向视野

- 向后看：分析过去的 designs、workloads、upsides 和 downsides，理解已有方案的取舍。
- 向前看：设想未来应用和未来技术，推动 state of the art，评估新的 design choices。
- 向上看：理解重要 problem domain，例如 medicine 和 genomics，判断系统应服务哪些人类需求。
- 向下看：理解 circuits、devices 和底层 technology，因为这些是建造体系结构的“材料”。

这四个方向共同定义了架构师的工作边界：架构师既要理解问题，也要理解实现材料；既要学习过去，也要为未来下注。

向后看对应 precedent。00:11:38–00:11:56，讲者说架构师需要理解过去设计的 tradeoffs、优点和缺点，也要理解过去的 workloads。这个能力让架构师避免重复低价值错误，并能解释为什么某些设计在某个时代有效。

向前看对应 innovation。00:11:56–00:12:27，讲者说架构师需要成为某种 dreamer，因为新体系结构创造的是此前没有存在过的东西。当有人提出新应用时，架构师不能只用今天的体系结构能力来封顶想象；如果所有新需求都被“今天做不到”挡住，innovation 就不会发生。

2.2 向上看：从 problem domain 出发

00:12:27–00:13:17，讲者解释 up 的含义：architecture sits in between problems and electrons。up 指向问题本身，架构师需要理解 important problems 的 nature，才能设计能够解决这些问题的系统。

他给出的具体例子是 genomics。00:12:52–00:13:10，讲者说，如果能够在 seconds 内快速分析 human genome，就可能在 medicine 中带来革命性影响。这里的重点是 problem-domain under-

⁴视频画面时间区间：00:11:11–00:12:30。若图像为裁剪图，时间区间仍对应原视频画面。

standing: 如果不了解 genome sequence analysis 的计算结构、数据规模和瓶颈, 架构师很难设计出真正有用的加速体系结构。

workload 是具体运行行为

workload 指系统实际运行的程序、输入和行为模式。讲者在 00:12:52–00:13:10 用 genome sequence analysis 说明 workload 可以来自医学问题; 在 00:14:42–00:15:04 又用 Facebook 风格的社交 workload 说明流行应用会随时间变化。体系结构设计要服务 workload, 因此 workload 的变化会直接改变缓存、内存、并行性和加速器设计的价值。

2.3 向下看: 技术是材料, 也是边界

00:13:20–00:13:52, 讲者把 circuits 和 device technology 比作建造 architecture 的 bricks。这个比喻很直接: 如果底层技术无法支撑某个设想, 体系结构就无法稳定落地。比如一个设计可能需要更高带宽、更低延迟或更低能耗的 memory technology; 如果实际器件做不到, 设计就会停留在纸面。

讲者还提醒, technology 之间有不同擅长点, 课程会特别讨论 memory technologies。这里埋下了后续 DRAM、refresh、RowHammer 等内容的入口: 底层物理和器件行为会向上影响系统性能、可靠性和架构策略。读者需要提前意识到, 体系结构最终要落在真实器件之上, 软件接口和数学模型都要接受器件边界约束。

常见误解: 把预测当成趋势口号

00:13:52–00:15:07 的预测讨论非常具体。架构师预测的对象包括 technology 和 workloads, 并且预测会进入真实设计承诺: 平台何时部署、生命周期多长、未来运行什么程序、底层器件能提供什么能力。空泛地说“未来会变化”没有工程价值; 工程价值来自把变化转成设计余量、可配置性、接口选择和风险管理。

2.4 预测与适应: 平台寿命把今天的选择推向未来

00:13:52–00:14:27, 讲者指出 leading architect 必须 predict and adapt to the future of technology, 因为设计的平台通常不会在明天立即使用。最激进的公司也可能在 6 months 后使用; 一个全新系统可能 5 years 后才投入使用, 并在 5 到 25 years from today 的时间跨度内发挥作用。

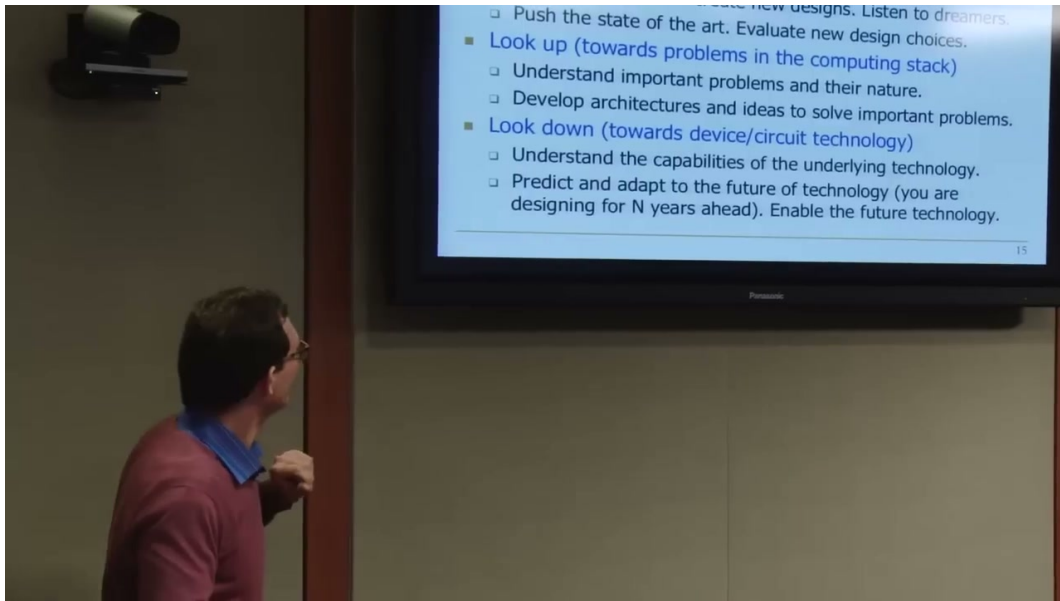


图 5: 预测和适应是架构工作的核心: 系统寿命跨越多年, 工作负载和技术条件会持续变化。⁵

这段话把体系结构设计变成一个带时间跨度的承诺。可以把设计视野写成:

$$H = t_{\text{deploy}} + t_{\text{life}}$$

- H : 架构师实际需要覆盖的未来时间范围。
- t_{deploy} : 从今天的设计决策到平台真正投入使用的时间。
- t_{life} : 平台投入使用后的生命周期。

讲者没有在幻灯片上给出这个公式; 它是对 00:14:00–00:14:27 时间跨度讨论的整理。公式提醒读者: 如果 H 很长, 今天的 workload 和 technology 观察只能作为起点, 不能直接等同于未来环境。

00:14:27–00:15:07, 讲者用 Facebook 举例说明 workload 预测风险。今天流行的社交应用不一定是 20 年后的主导 workload。类似地, 今天看来合理的 memory hierarchy、accelerator mix 或 parallelization strategy, 也可能因为应用结构、数据规模和器件能力变化而失去优势。

预测失败会变成系统级成本

体系结构设计通常很难在部署后彻底重做。00:15:45–00:16:10, 讲者把建筑和计算机平台作类比: 建筑可以修改, 很多体系结构设计在部署后难以修改, 除非一开始就设计成 configurable 或 reconfigurable。因此, 预测能力会直接影响系统长期价值; 预测得太窄, 平台会被未来 workload 和 technology 变化拖住。

2.5 工程分析与判断力要一起工作

00:15:07–00:16:31, 讲者给出 takeaways: 成为架构师很难, 需要同时考虑 workload、technology、接口、成本、功耗、可靠性和可编程性, 并且需要对 ideas and tradeoffs 有 good intuition and insight。他还说 architecture 带有超出精确科学模型的部分, building architecture 有 art 的部分, computer architecture 也有类似性质, 甚至可能更加复杂。

这里的“art”不表示随意发挥。它指的是在信息不完整、目标冲突、未来不确定的情况下, 架构师仍要使用经验、模型、实验、直觉和判断来做取舍。工程分析提供数据边界, 例如延迟、吞吐、面积、功耗、可靠性; 判断力负责在多个目标之间选择优先级。课程后续的 memory controller、DRAM refresh、RowHammer 等案例, 会不断展示这种分析与判断的组合。

⁵视频画面时间区间: 00:13:52–00:15:07。若图像为裁剪图, 时间区间仍对应原视频画面。

直觉需要被机制校准

good intuition 在 00:15:19–00:16:19 被反复强调，但直觉不能脱离机制。一个听起来合理的策略，放到共享内存、缓存一致性或 DRAM 调度中可能带来反效果。体系结构训练的目标之一，就是让直觉不断接受具体机制和测量结果的校准。

2.6 为什么 architectural innovation 能改变日常系统

00:16:31–00:17:46，讲者把架构师的难度转成课程的价值：许多科学和日常生活中的 innovation，都依赖 architectural innovation enabled very high-performance systems。学生今天习以为常的设备，比 10 年或 20 年前的系统更强大，这背后有一系列体系结构创新支撑。



图 6: 现代系统能力来自一系列架构创新，课程随后会把这些创新拆解成可学习的原则。⁶

讲者列出的例子包括 pipelining、caches、locality exploitation、out-of-order execution、memory system innovations 和 memory controllers。这些名词在第一讲此处只是预告，已经足以说明课程主线：现代系统的能力来自一系列围绕 workload、technology 和 tradeoff 的组织创新，单个元件速度提升只覆盖其中一部分。

几个后续会反复出现的创新词

- pipelining: 把执行过程分成阶段，让多个指令在不同阶段并行推进，提高整体吞吐。
- cache: 利用 locality，把近期或相邻可能访问的数据放到更快的存储层次中。
- out-of-order execution: 在依赖关系允许时改变指令实际执行顺序，让硬件更好地利用等待时间。
- memory controller: 观察内存请求队列并安排 DRAM 操作，是后续共享资源和公平性讨论的关键角色。

这些机制都体现了同一件事：架构创新把对 workload 和 technology 的理解固化成系统组织方式。

⁶视频画面时间区间：00:16:31–00:17:46。若图像为裁剪图，时间区间仍对应原视频画面。

2.7 本章小结

本节用 00:11:11-00:17:46 定义了 computer architect 的工作：向后理解 precedent，向前预测 workload 和 technology，向上理解 problem domain，向下理解 circuits 与 devices。架构师设计的平台会跨越 6 个月、5 年甚至 25 年的时间范围，因此预测直接构成设计本身。体系结构也需要工程分析与判断力共同工作；pipelining、caches、locality exploitation、out-of-order execution、memory system innovations 和 memory controllers 等创新，正是这种能力在现代系统中的具体成果。

3 作为硬件-软件接口的体系结构

3.1 为什么要从“中间层”看计算机

前面两节已经把“建筑师”的角色讲成一种面向未来、面向约束的设计能力。进入计算机体系结构本身时，课程先把学生放在两个已经熟悉的视角之间：一边是程序员看到的计算机系统，另一边是硬件设计者看到的数字逻辑。程序员通常从 C、汇编、运行时、操作系统等接口理解系统；硬件设计者则从门电路、连线、时序和具体硬件结构理解系统。体系结构课要处理的正是两者之间的转换地带。

这个“中间层”承担一个具体问题：任意程序怎样最终变成数字逻辑上的执行？从程序员写下的意图，到硬件中门电路和导线的动作，中间必须经过一组明确的接口、约定和实现选择。架构师或微架构师的工作，就是设计这些选择，使计算机达到给定的系统目标，例如性能、能耗、可编程性、可靠性、成本和可扩展性。

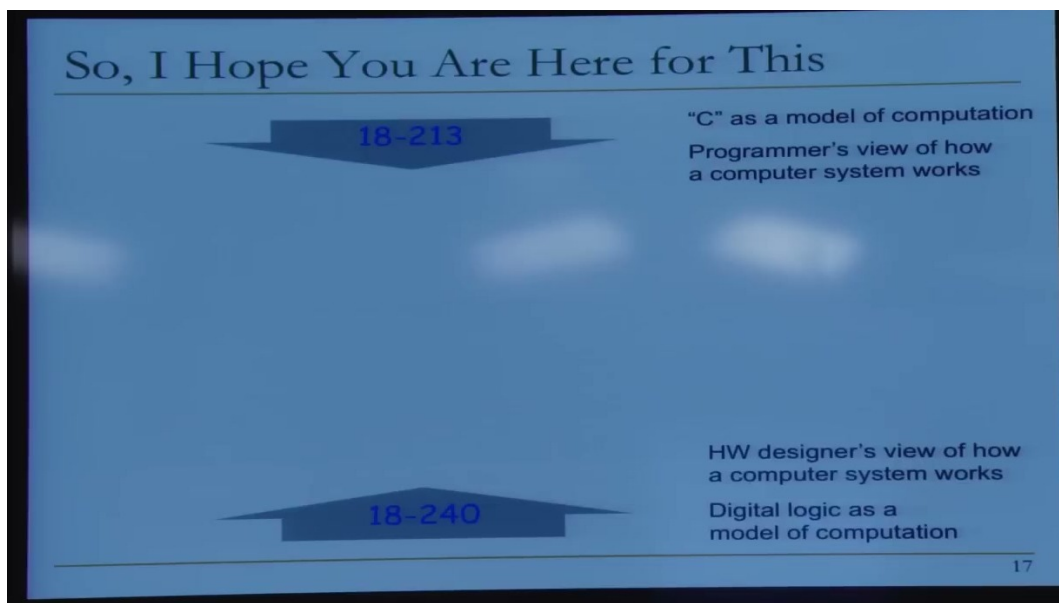


图 7：计算机体系结构位于软件需求和硬件机制之间，程序员视角与微架构视角需要同时进入设计判断。⁷

核心定位：体系结构位于接口处

计算机体系结构关注软件意图怎样被硬件机制承接。它既要理解上层程序要解决的问题，也要理解下层技术能够提供的机制；架构师的决策会同时改变程序员的成本和硬件实现者的成本。

⁷视频画面时间区间：00:17:47-00:19:07。若图像为裁剪图，时间区间仍对应原视频画面。

这个定义很容易被误读成“体系结构就是更高级的硬件课”。讲者特意压住这种理解：实验中会设计硬件，课程也会进入 ISA、微架构、逻辑和模拟器；但它的核心视角是硬件-软件接口。接口这个词很关键。接口像合同：上层按照合同使用下层，下层按照合同提供行为。合同写得太窄，上层难以表达需求；合同暴露太多，下层难以保持实现自由，程序员也会被细节淹没。

3.2 程序员视图、硬件设计视图与架构师视图

程序员视图强调“我能写什么、我能假设什么、我能观察什么”。例如，高级语言程序员关心数据结构、控制流、库、运行时行为；汇编程序员还会关心寄存器、指令和寻址方式。硬件设计视图强调“这个行为怎样由电路稳定、快速、节能地实现”。例如，一个加法、一次访存、一次分支预测、一次缓存命中，都对应底层结构和时序。

架构师视图需要把这两组问题放在同一张桌面上。一个设计在硬件侧看起来规整、容易扩展，可能在软件侧制造并行化、调试或性能调优的负担；一个设计在软件侧看起来方便，可能要求硬件付出面积、功耗或验证复杂度。架构师的难处正在这里：他需要拥有跨层可见性，知道上层真正需要什么，也知道下层真实能承受什么。

常见误解：只看单侧会低估设计代价

如果只从硬件侧评价一个设计，很容易把“容易制造”“容易扩展”当作充分目标；如果只从软件侧评价一个设计，又容易忽略面积、功耗、时钟频率、可靠性和验证成本。体系结构的判断需要把两侧目标同时放进约束集合。

这种跨层判断可以用一个朴素类比理解：程序员像提出运输需求的人，硬件像道路、桥梁和车辆，体系结构像交通规则和道路规划。规则设计得好，使用者不必知道每一块沥青怎样铺设，也能高效抵达目的地；规则设计得粗糙，使用者会被迫绕路、等待或承担额外协调成本。

3.3 多核系统：硬件选择怎样改变软件成本

讲者用多核系统举了第一个具体例子。现代芯片经常放入多个处理器核心。对硬件设计者来说，把多个相对简单且规则的核心连接起来，通常比把全部芯片面积都堆到一个极其强大的单核上更可控：设计复杂度更低，结构更规整，功耗和频率压力也更容易管理。

这项选择把一部分成本移动到了程序员一侧。单线程程序在一个极强单核上可以直接跑得很快；多核系统要真正发挥全部核心，程序员、编译器、运行时或库就需要把工作拆成可以并行执行的部分。并行化牵涉任务分解、同步、通信、负载均衡、共享数据保护和调试复杂度。硬件没有消失，软件也没有独立胜出，设计选择改变了成本分布。

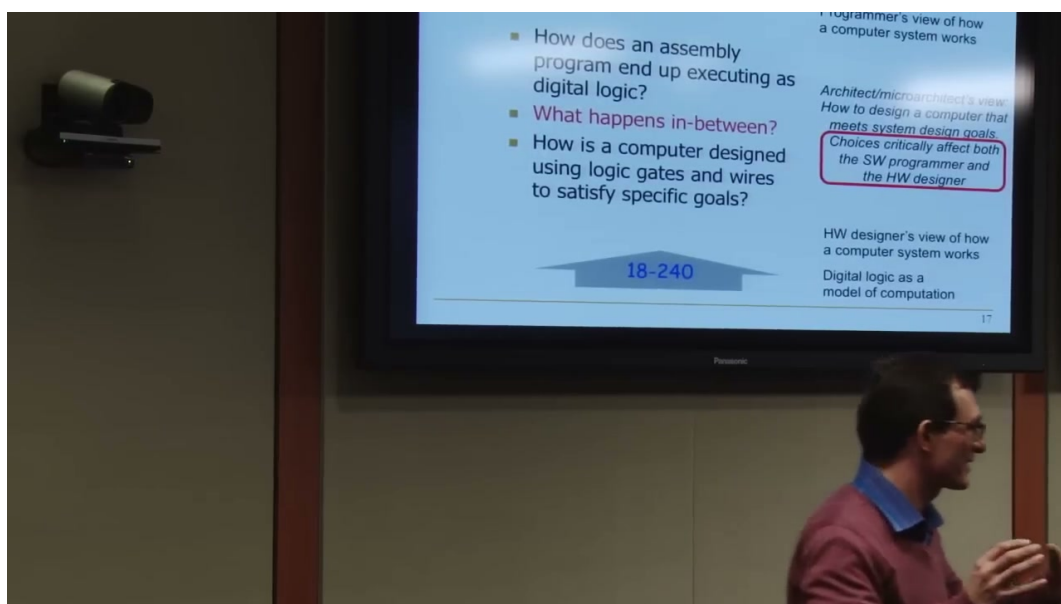


图 8: 多核把并行化责任部分推给程序员, 也让共享硬件资源成为软件性能的一部分。⁸

背景补充: 多核为什么成为典型接口案例

多核设计展示了体系结构决策的双向影响。硬件侧获得规则复制和总体吞吐能力, 软件侧需要显式或隐式地发掘并行性。这个案例也为后面的 DRAM 共享资源干扰铺路: 多个核心一旦共享缓存、内存控制器和 DRAM bank, 程序之间的影响就会从“谁在运行”扩展到“谁在占用底层资源”。

这个例子还提醒读者: 接口设计经常带有时间维度。架构师可能为了未来的工作负载、工艺条件和系统规模做选择; 程序员则在今天面对并行化工具、语言模型和调试能力的限制。一个设计在长期技术路线中合理, 在短期软件生态中仍然可能昂贵。好的体系结构分析要同时问两个问题: 这项选择让硬件获得了什么? 它要求软件付出什么?

3.4 实验中的硬件与课程真正关注的边界

讲者随后明确说明: 实验会要求学生设计硬件, 但这门课的知识重心是硬件-软件接口。甚至为了设计硬件, 学生还要写大量模拟器; 模拟器本身就是软件, 用来描述、测量和比较硬件机制。这个安排本身就是课程主题的缩影: 软件工具帮助理解硬件, 硬件机制决定软件行为。

⁸视频画面时间区间: 00:20:30-00:22:03。若图像为裁剪图, 时间区间仍对应原视频画面。

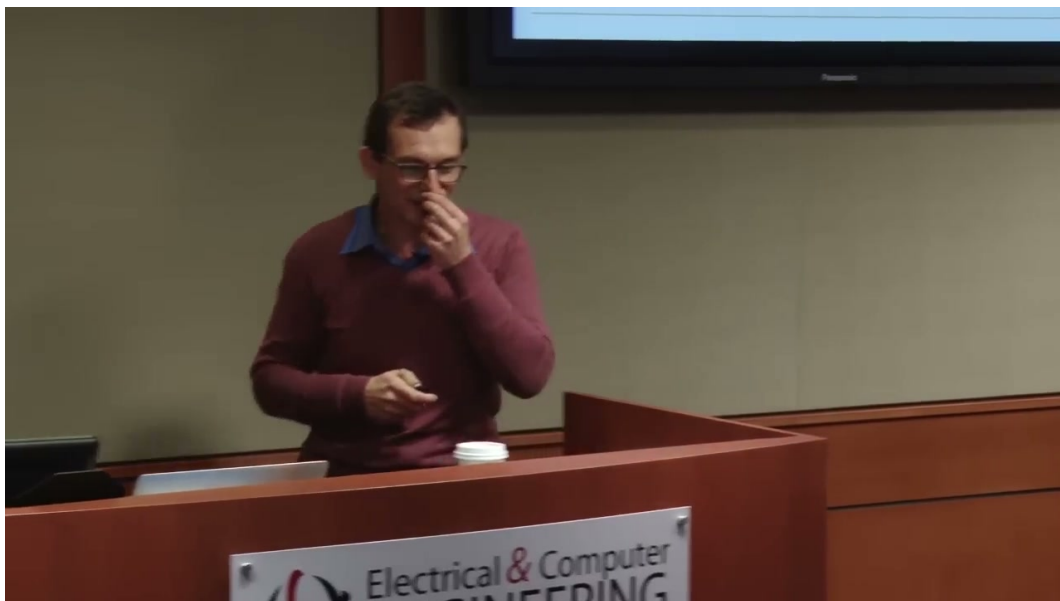


图 9: 硬件实验服务于接口理解: 学生需要看见硬件决策如何改变软件可见的成本。⁹

本节带走的判断方式

遇到任何体系结构机制时, 读者都应同时追问三件事: 上层程序能看到什么接口? 下层硬件怎样实现这个接口? 这个实现选择怎样改变性能、能耗、可编程性和可靠性?

3.5 本章小结

本节把计算机体系结构定位为程序员视图和硬件设计视图之间的接口学科。架构师需要理解软件要解决的问题, 也需要理解逻辑、电路和技术趋势能提供的实现空间。多核系统说明, 一个看似硬件侧自然的设计选择, 会把并行化、同步和调试成本带给软件层。实验会触及硬件设计, 课程的核心训练则是跨越硬件与软件边界做判断: 接口暴露什么、隐藏什么、由谁承担代价、最终系统目标是否真正被满足。

4 转换层级与抽象的力量

4.1 计算的目的是: 从问题到洞见

讲者借 Richard Hamming 的一句话开启这一段: “计算的目的在于洞见。”这句话给体系结构加上了方向感。计算机服务于人类问题, 例如根据病史和基因信息寻找合适药物、比较基因序列、分析大规模数据, 或者把某种现实需求转化成可执行的信息处理任务。

问题在于, 人类问题和电子运动之间隔着巨大的表达距离。人可以说“根据这个人的基因找出可能有效的药物”, 电子只能在电路中移动、存储电荷、形成电压差。体系结构课程要解释的, 就是这段距离怎样被一层层接口缩短。

⁹视频画面时间区间: 00:22:03–00:22:46。若图像为裁剪图, 时间区间仍对应原视频画面。

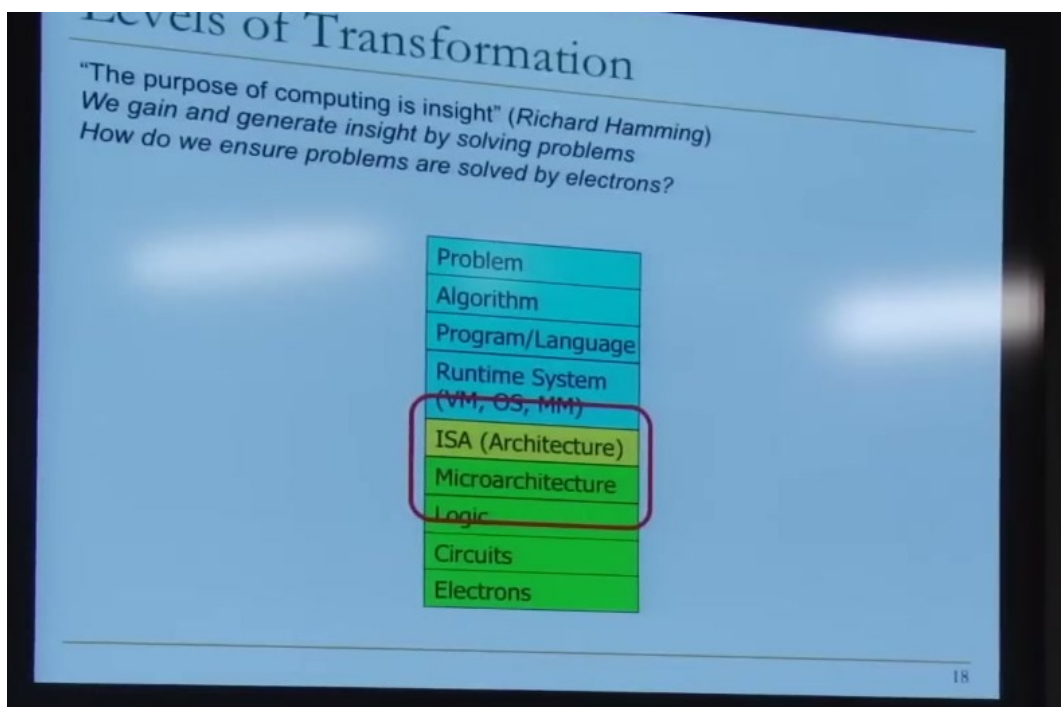


图 10: 从问题、算法、程序到 ISA、微架构、电路和电子，计算是一条逐层翻译链。¹⁰

转换层级的主线

计算把高层问题逐步翻译成物理状态变化：问题被整理成算法，算法被写成程序，程序通过运行时、操作系统、虚拟机和编译器变成指令，指令由 ISA 定义，微架构实现 ISA，微架构由逻辑门组成，逻辑门由电路实现，电路最终依赖物理规律和电子运动。

这条链条的每一段都是接口。算法接口让程序员不必直接谈电压；ISA 让软件可以面对一套稳定的指令和状态；微架构让同一套 ISA 可以由不同硬件组织实现；逻辑和电路接口让设计可以逐步下降到物理层。读者可以把它想象成一条翻译链：每个译者负责保留上一层的关键含义，同时把表达方式改成下一层能够执行的形式。

4.2 从问题到电子：接口怎样组织复杂性

这一段的重点不只是列出层级名称，还要理解层级为什么存在。直接让人和电子“对话”没有现实接口。人类需要的是问题、数据、约束和答案；电子提供的是电荷、开关、延迟和能耗。中间层级把不可管理的复杂性切成可管理的局部合同。

典型转换链可以按如下顺序理解：

1. 问题：例如比较两段基因序列，或者从病史和基因中寻找治疗线索。
2. 算法：把问题变成明确步骤，例如相似性比较、动态规划、搜索或统计推断。
3. 程序：用高级语言、库和数据结构表达算法。
4. 运行时与系统软件：由运行时、操作系统、内存管理器或虚拟机管理执行环境。
5. **ISA**：定义程序可见的指令、寄存器、内存访问和架构状态。
6. 微架构：用流水线、缓存、预测器、执行单元和调度逻辑实现 ISA。
7. 逻辑、电路与物理：用门电路、晶体管、电压、电荷和电子运动完成实际计算。

¹⁰视频画面时间区间：00:22:47–00:25:19。若图像为裁剪图，时间区间仍对应原视频画面。

ISA 在这里尤其关键。它是软件和硬件之间的程序可见合同。软件通过 ISA 表达“要执行哪些操作”，硬件通过微架构决定“怎样高效执行这些操作”。同一 ISA 可以有多个微架构实现：有的追求高性能，有的追求低功耗，有的追求面积效率。这种分离让软件生态可以稳定，同时给硬件创新留下空间。

边界也会移动

层级边界来自工程选择。编译器指令调度可以被看成语言层、运行时层或 ISA 附近的优化；虚拟机可以隐藏真实 ISA；某些硬件控制信号也可能被提议暴露给编译器。边界的移动会改变谁拥有优化机会，也会改变谁承担复杂性。

4.3 Hamming distance：一个形式化接口的微型例子

讲者随后转向 Hamming distance。先用直觉说，它衡量两个等长字符串在多少个对应位置上不同。比较时要把两个字符串按位置对齐，然后逐格检查：相同的位置贡献 0，不同的位置贡献 1，最后把这些贡献相加。

更一般地，若两个字符串长度相同，并且都来自同一个符号表，Hamming distance 可以写成：

$$d_H(x, y) = \sum_{i=1}^n \mathbf{1}[x_i \neq y_i]$$

符号逐项说明如下：

- $d_H(x, y)$ ：字符串 x 与字符串 y 之间的 Hamming distance。
- x, y ：待比较的两个字符串。
- Σ ：字符串所使用的符号表，例如二进制符号表 $\{0, 1\}$ 。
- $x, y \in \Sigma^n$ ： x 和 y 都由 Σ 中的符号组成，并且长度都是 n 。
- n ：每个字符串的长度。
- i ：位置索引，从 1 到 n 。
- x_i ：字符串 x 在第 i 个位置上的符号。
- y_i ：字符串 y 在第 i 个位置上的符号。
- $\neq$ ：两个符号不同。
- $\mathbf{1}[\cdot]$ ：指示函数；括号内条件成立时取 1，条件不成立时取 0。
- $\sum_{i=1}^n$ ：把所有位置上的不匹配指示值相加。

例子很小，但足够说明定义：1101 和 1100 都有四个二进制位，前三位相同，最后一位不同，所以 Hamming distance 等于 1。这个距离不局限于二进制数；只要两个字符串等长、位置对齐、符号表一致，就可以逐位计数。

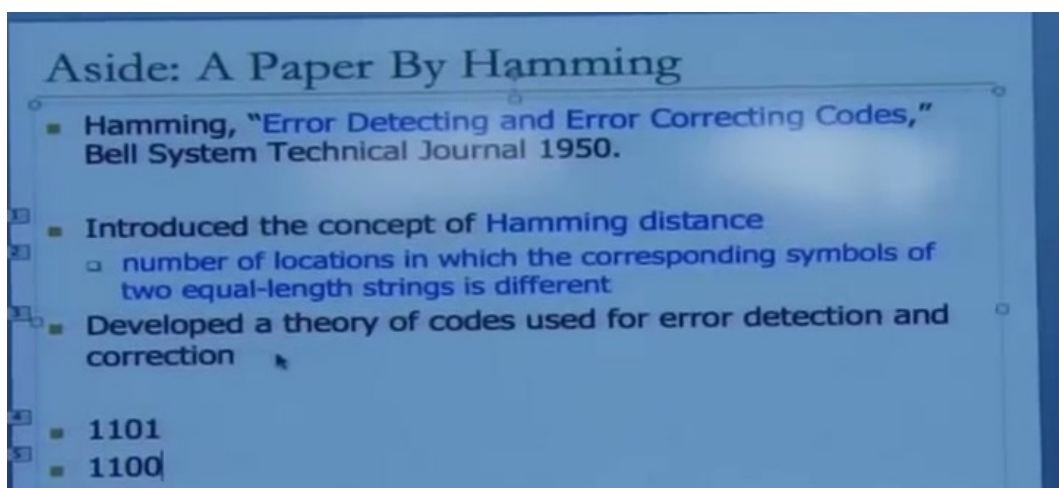


图 11: Hamming distance 用“逐位数差异”的形式展示了抽象定义如何支持可靠性机制。¹¹

为什么这个小定义重要

Hamming distance 把“差多少”变成可计算的精确定义。这个定义支撑错误检测和错误纠正：如果合法编码彼此之间间隔得足够远，传输或存储中的少量符号错误就更容易被发现，并且在条件允许时可以推回原来的合法编码。

这里的教学价值在于：一个清晰的形式化接口可以放大工程能力。定义本身只是在数位置差异；一旦把它放入编码理论，系统就能对噪声、存储错误和传输错误做出可证明的处理。后面内存可靠性章节会继续展示同一种思想：硬件世界里会发生错误，体系结构需要用合适的接口和冗余机制把错误控制在系统可承受范围内。

4.4 Hamming 的研究建议：想法需要被清楚表达

讲者短暂提到 Hamming 的研究经验：伟大的想法需要被别人理解、评价和复用。写作和演讲直接参与研究影响力的形成；它们决定一个思想能否产生影响。这个旁支和本课主题有内在联系。体系结构是一门跨层学科，跨层设计需要不同背景的人共同理解同一套机制：程序员、编译器作者、操作系统研究者、电路设计者和硬件验证人员都要看懂接口承诺与实现限制。

研究沟通与工程接口

清楚表达思想，和清楚定义接口，是同一种工程习惯在不同场景中的表现。含糊的论文让读者无法复现贡献；含糊的 ISA 或微架构说明会让软件、硬件和验证工作同时变脆。好设计通常伴随好解释。

4.5 抽象带来的生产力

转换层级自然产生抽象。抽象的收益很直接：上层只需要知道下层的接口，无需知道下层完整实现。软件工程师写 Java 程序时通常无需知道电路怎样实现加法；C 程序员通常无需知道每条指令怎样编码成二进制；算法设计者通常无需知道缓存替换策略。抽象让人把注意力集中在当前层的目标上，从而提高生产力。

可以把抽象理解成一组“默认屏蔽”。它屏蔽电路细节，让程序员能写业务逻辑；屏蔽微架构细节，让二进制程序可在同一 ISA 的不同处理器上运行；屏蔽物理细节，让逻辑设计可以用门和时序讨论系统行为。没有这些屏蔽，现代软件和硬件规模都会失控。

¹¹ 视频画面时间区间：00:25:33–00:28:20。若图像为裁剪图，时间区间仍对应原视频画面。

抽象的正面作用

抽象通过隐藏低层细节来换取生产力、可移植性和模块化。它让高层工作者依赖稳定合同，把复杂系统拆成可设计、可验证、可替换的层。

4.6 抽象的代价：可见性不足会阻碍优化

抽象也会制造盲点。只要系统行为符合预期，盲点通常可以接受；一旦出现性能、能耗、正确性或可编程性问题，缺乏跨层可见性就会变成真实障碍。

讲者给出的程序性能例子很典型：程序员看到自己的代码运行很慢，手里能改的只是源代码和数据结构。若他不知道底层缓存行为，就可能错过关键原因：一个巨大数据结构造成大量 cache miss；把数据分块，让程序反复处理能装进缓存的小块，性能也许会显著改善。问题表面在程序，机制却藏在内存层级里。

同样的风险也存在于硬件侧。硬件设计者如果不了解软件使用方式，可能设计出难以编程、难以优化或难以被生态采用的机制。一个处理器即使在局部指标上很漂亮，只要让多数程序员和编译器无法有效使用，它的系统价值就会受损。

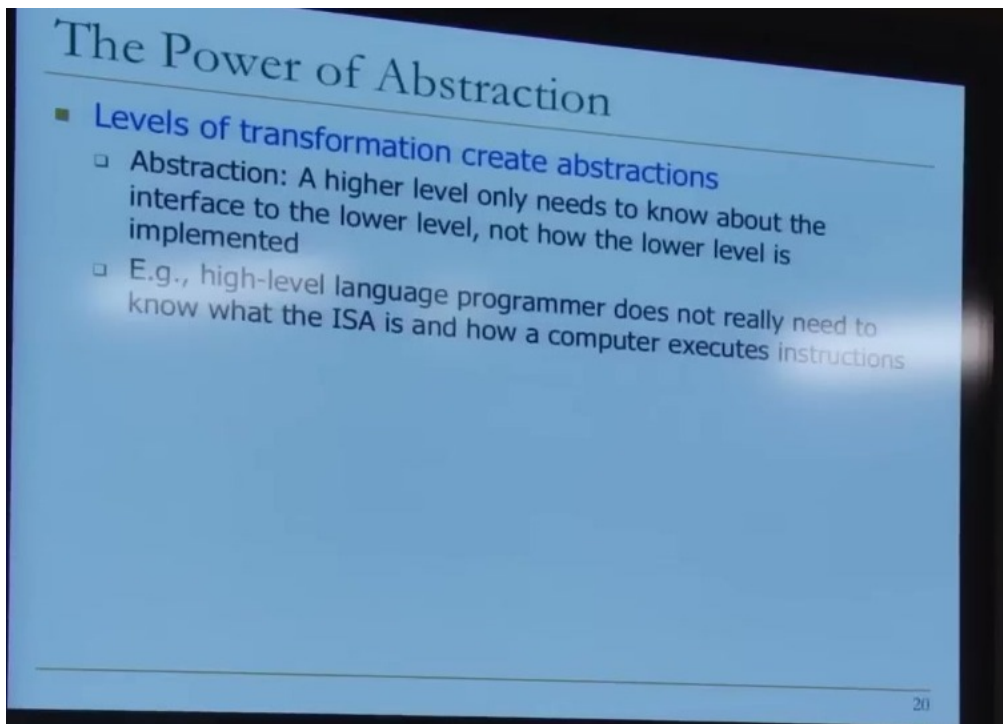


图 12: 抽象隐藏细节以提高生产力，也会遮住性能、能耗和可靠性的真实来源。¹²

抽象的“未知的未知”

抽象隐藏细节，也会隐藏问题来源。程序变慢时，原因可能在算法、数据布局、缓存、编译器、操作系统调度或硬件资源冲突；硬件效率低时，原因也可能来自软件访问模式。架构师需要受控地穿透抽象层，找到真正起作用的机制。

因此，体系结构课程训练的能力侧重于有选择地穿透抽象。它训练读者知道何时尊重抽象、何时跨越抽象、跨越以后应该看哪些信号。过度暴露底层细节会破坏生产力；过度依赖高层接口会错过性能和可靠性事实。架构师要在这两种风险之间选择合适的可见性。

¹²视频画面时间区间：00:30:28–00:31:12。若图像为裁剪图，时间区间仍对应原视频画面。

4.7 本段课程目标：理解平台行为，做跨层决策

讲者最后把这段内容收束成两个课程目标。第一，理解处理器或平台在软件层之下怎样工作，以及硬件决策怎样影响软件和程序员。第二，让学生能够做跨越不同层级和系统组件的设计与优化决策。

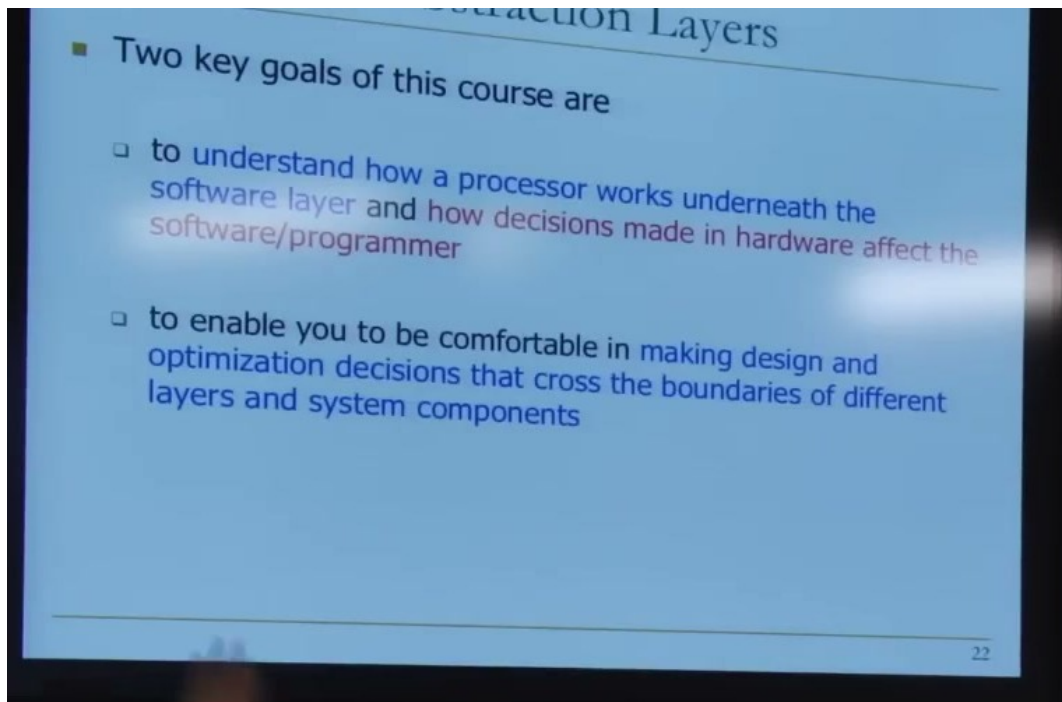


图 13: 课程目标聚焦于理解平台行为，并据此做出跨层设计与优化决策。¹³

这两个目标互相支撑。不了解平台内部行为，就无法判断性能瓶颈来自哪里；缺乏跨层决策能力，就只能在熟悉的一层做局部修改。后面的多核内存干扰、DRAM refresh 和 RowHammer 都会继续强化同一件事：真实系统的关键现象经常跨越软件、体系结构、微架构、电路和物理层。

本节的最终抓手

抽象让系统可建造，跨层理解让系统可优化、可诊断、可长期演进。计算机体系结构的核心训练，是在稳定接口和底层机制之间来回切换，并用这种切换做出更好的设计判断。

4.8 本章小结

本节从“计算的目的在于洞见”出发，说明高层问题必须经过一系列转换层级才能变成电子层面的物理状态变化。问题、算法、程序、运行时、ISA、微架构、逻辑、电路和物理共同构成一条翻译链，其中 ISA 是软件可见的关键合同。Hamming distance 展示了清晰形式化定义的力量：一个“逐位数差异”的概念可以支撑错误检测和错误纠正。抽象带来生产力和模块化，也会遮蔽性能、能耗、正确性和可靠性的真实原因。课程接下来的核心训练，就是让读者理解平台内部行为，并能在不同层级之间做设计与优化决策。

¹³视频画面时间区间：00:35:04–00:35:34。若图像为裁剪图，时间区间仍对应原视频画面。

5 案例一：多核减速、DRAM bank 与调度公平性

5.1 从一个反直觉的减速现象开始

这一段课从一个多核系统图进入：多个 core 各自执行程序，旁边有私有或共享 cache，后面连着 memory controller，再连到 DRAM。讲者强调这些部件后面都会在课程中展开，这里先抓住一个足够尖锐的问题：当两个看似互相独立的单线程程序分别跑在两个 core 上时，它们为什么会让对方变慢，而且变慢程度差异巨大？

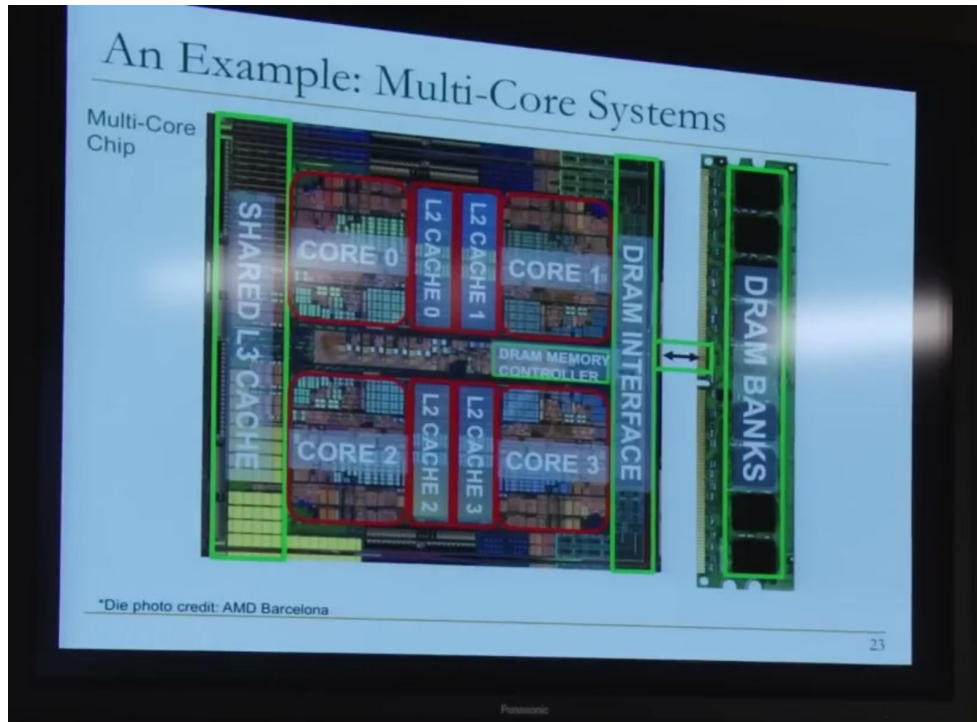


图 14: 多核芯片中的核心、缓存、内存控制器与 DRAM bank：共享内存资源是后续性能干扰的根源。¹⁴

实验设置很朴素。把 MATLAB 放在一个 core 上，把 GCC 放在另一个 core 上；再分别测量 MATLAB 单独运行、GCC 单独运行时的性能；最后把共同运行时的运行时间与单独运行时的运行时间相除。结果很不对称：MATLAB 与 GCC 一起运行时大约慢 7%，GCC 与 MATLAB 一起运行时大约慢 3×。这意味着 GCC 受到的伤害远大于 MATLAB。

¹⁴视频画面时间区间：00:35:59–00:36:42。若图像为裁剪图，时间区间仍对应原视频画面。

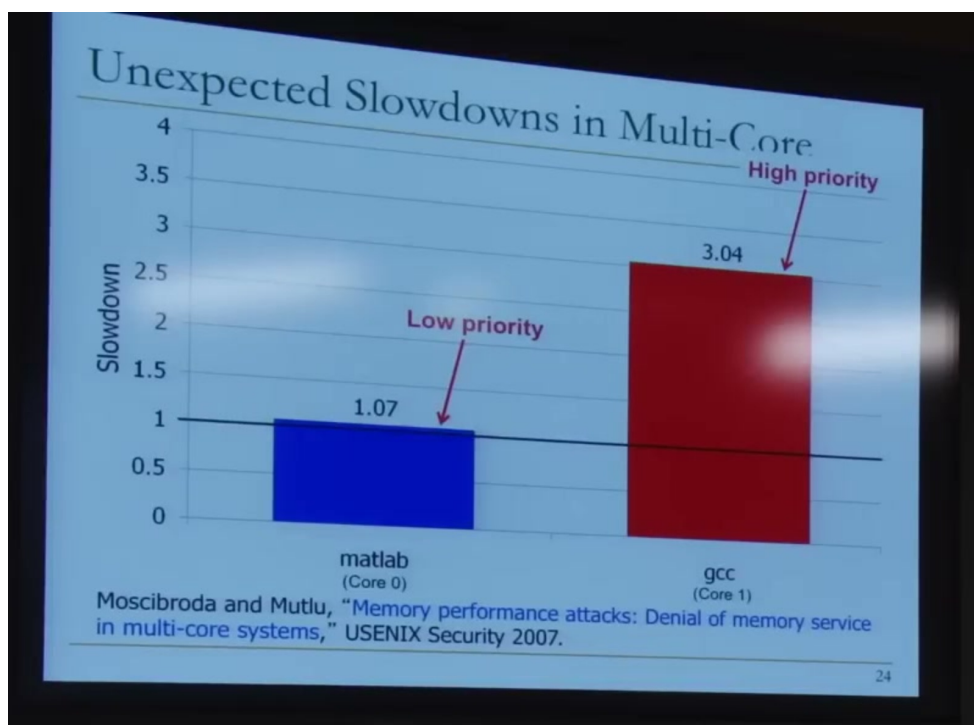


图 15: MATLAB 与 GCC 在两个核心上同时运行时出现非对称减速: 低优先级 MATLAB 约 1.07 倍, 高优先级 GCC 约 3.04 倍。¹⁵

为了把这个现象变成可计算的问题, 需要先定义 slowdown。直观地说, slowdown 衡量一个程序在共享系统中比独占系统中慢了多少; 分子是共享执行时的运行时间, 分母是单独运行时的运行时间。

$$\text{slowdown}_i = \frac{T_i^{\text{shared}}}{T_i^{\text{alone}}}$$

- i : 被衡量的应用程序编号, 例如 MATLAB 或 GCC。
- slowdown_i : 应用 i 的减速倍数。
- T_i^{shared} : 应用 i 与其他应用共同运行时的运行时间。
- T_i^{alone} : 应用 i 独占同一系统运行时的运行时间。
- shared: 共享执行环境, 多个应用同时占用某些硬件资源。
- alone: 单独执行环境, 用作基准线。

若 $\text{slowdown}_i = 1$, 共享执行没有让应用变慢; 若 $\text{slowdown}_i = 1.07$, 运行时间增加约 7%; 若 $\text{slowdown}_i = 3$, 运行时间变成独占时的三倍。MATLAB/GCC 案例的关键就在这里: 两个程序都在运行, 两个 core 都在工作, GCC 却像在等待一个看不见的瓶颈。

slowdown 是共享资源伤害的应用级度量

单看 memory controller 完成了多少请求, 只能看到硬件吞吐; 看 slowdown, 才能看到某个应用相对于独占运行被伤害了多少。公平性讨论需要这种应用级度量, 因为用户关心的是程序完成时间, 服务系统关心的是服务质量, 攻击者也可能利用这种差异制造拒绝服务效果。

¹⁵视频画面时间区间: 00:36:56–00:38:03。若图像为裁剪图, 时间区间仍对应原视频画面。

5.2 为什么 OS priority 没有修好它

一个自然猜想是操作系统调度器偏心了。Linux 里可以用类似 `nice` 的机制降低 MATLAB 的调度优先级，让 GCC 获得更高优先级。讲者说他们做了这个实验，结果 slowdown 基本没有变化。这个结果很有教学价值：CPU 调度优先级控制的是进程被安排到 core 上执行的机会；在这个实验中，两个单线程应用已经分别占据两个可用 core，OS 把它们放上去以后，后面的 DRAM 请求排队由 memory controller 接管。

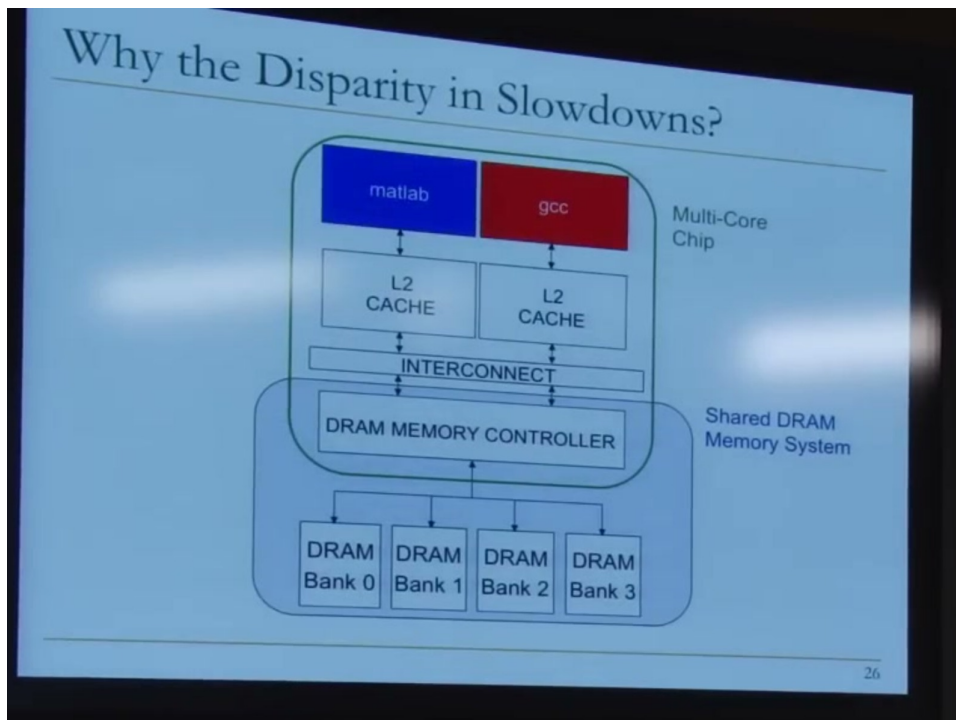


图 16: 两个应用拥有各自的 L2 Cache，却汇入同一个 DRAM 控制器和多个 Bank；竞争点发生在更低层。¹⁶

这并不表示 OS 在所有场景下都无能为力。若进程数量多于 core 数量，OS priority 会影响谁更常获得 CPU 时间；若系统软件能控制物理页映射、channel 映射或 bank 映射，它也可能绕开一部分干扰。这个案例中的局限更具体：memory controller 的请求队列里同时出现 MATLAB 和 GCC 的内存请求，而常规 OS priority 信息没有直接进入这个队列调度逻辑。

常见误解：把 CPU 调度优先级当成全系统优先级

`nice` 一类机制主要作用在 CPU 调度层。多核系统中，进程一旦已经在不同 core 上同时运行，它们发出的 cache miss 和 DRAM 请求会进入更低层的硬件队列。若 memory controller 不读取 OS 的优先级意图，CPU 层的优先级就很难约束 DRAM bank 中的服务顺序。

讲者让学生先猜原因。有人猜 cache locality，有人猜自动并行化，也有人提到 DRAM access 被 MATLAB 阻塞。cache 猜想在另一些系统里完全可能成立，因为共享 cache 也会造成干扰；但这个实验中两个 core 的 cache 是分开的，真正的冲突位置落在共享 memory controller 与 DRAM 访问路径上。

¹⁶ 视频画面时间区间：00:41:14–00:42:05。若图像为裁剪图，时间区间仍对应原视频画面。

5.3 DRAM bank、row 与 row buffer

为了理解 GCC 为什么会等很久，需要把 DRAM bank 的内部结构摊开看。一个 DRAM 系统包含多个 bank；一个 bank 可以想成二维单元阵列，有 row 和 column。访问 DRAM 时，不能直接从巨大阵列中随手取一个字节；DRAM 先把整行打开到 row buffer，然后再根据 column 地址从 row buffer 中取出需要的数据。

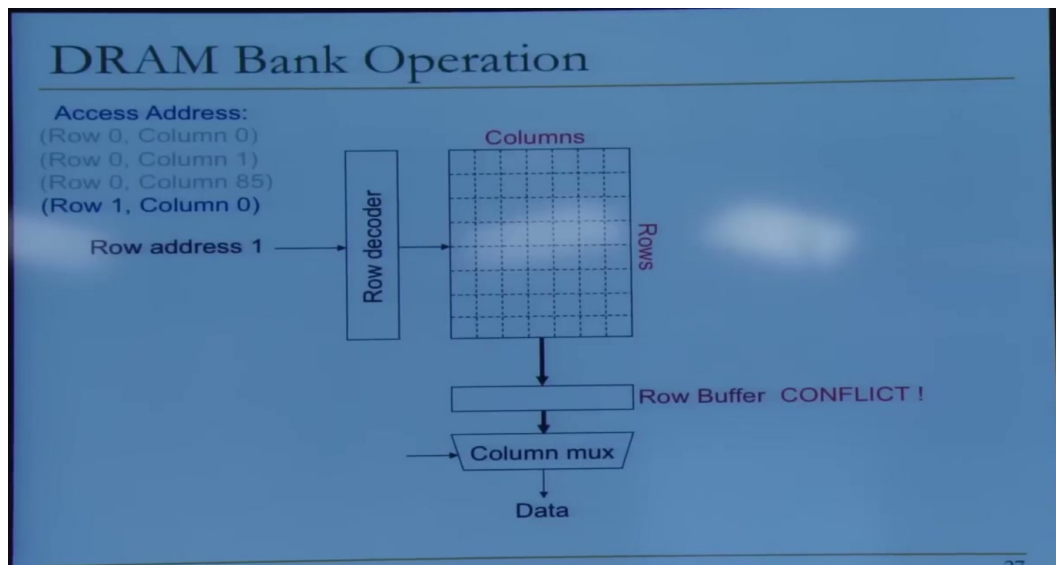


图 17: DRAM bank 的行缓冲冲突：访问另一个 row 时需要切换已打开的行，延迟明显高于 row buffer hit。¹⁷

第一次访问某个 row 时，memory controller 发送 row address，DRAM 执行 activate，把这一整行搬到 row buffer；随后 controller 再发送 column address，DRAM 从 row buffer 中选出对应列的数据返回。row buffer 因而像一张已经摊开的纸：纸已经在桌面上时，查同一页的另一个位置很快；换到另一页时，需要先收起当前页，再拿出新页。

row buffer 可以当作一个极小、极特殊的 **cache** 来理解

普通 cache 以 cache block 为基本单位保存最近使用的数据；row buffer 以 DRAM row 为单位保存当前打开的一整行。它没有复杂替换策略，当前 bank 通常只打开一个 row。请求命中当前打开的 row 时，访问路径短；请求落到另一个 row 时，必须经历关闭旧 row、打开新 row 的额外步骤。

这就引出两个术语。row buffer hit（行缓冲命中）指请求访问的 row 已经在 row buffer 里；controller 只需发 column address，延迟较低。row conflict（行冲突）指请求访问另一个 row；DRAM 需要先 precharge 当前 row，再 activate 目标 row，再访问 column，延迟明显更高。讲者给出的数量级是，当代 DRAM 中 row conflict 往往比 row buffer hit 慢约 2× 到 3×。

¹⁷ 视频画面时间区间：00:42:15–00:44:50。若图像为裁剪图，时间区间仍对应原视频画面。

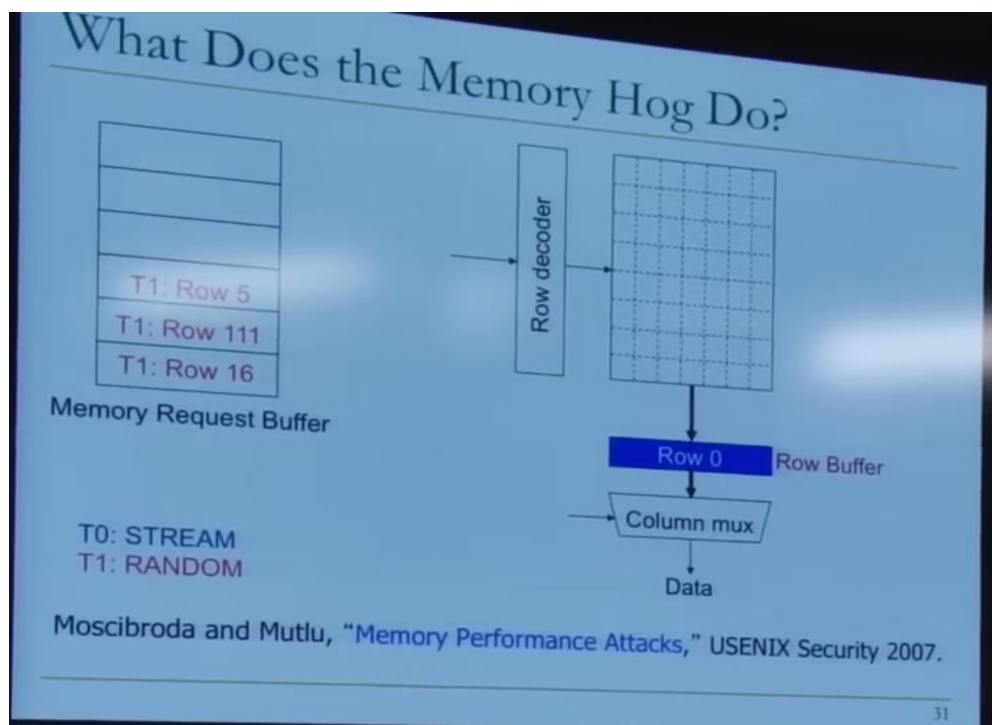


图 18: 内存请求缓冲区中的 RANDOM 请求排队等待, 而 Row 0 保持打开, STREAM 的 row buffer hit 被持续优先服务。¹⁸

row buffer hit 与 row conflict 的核心差别

row buffer hit 复用已经打开的 row, 省掉 activate 的主要开销; row conflict 需要关闭当前 row 并打开另一个 row。memory controller 若优先服务 row buffer hit, 就能提升 DRAM 单位时间内完成的请求数, 这也是后面公平性问题的起点。

5.4 FR-FCFS 风格调度: 局部高效如何放大成不公平

现代 memory controller 往往采用一种 FR-FCFS 风格的策略。FR-FCFS 可以展开为 first-ready, first-come-first-served: 先服务已经 ready 的请求, 在 DRAM 语境里通常就是 row buffer hit; 在同类请求之间, 再用进入队列的时间做 tie breaking, 老请求优先。

¹⁸视频画面时间区间: 00:48:39–00:49:56。若图像为裁剪图, 时间区间仍对应原视频画面。

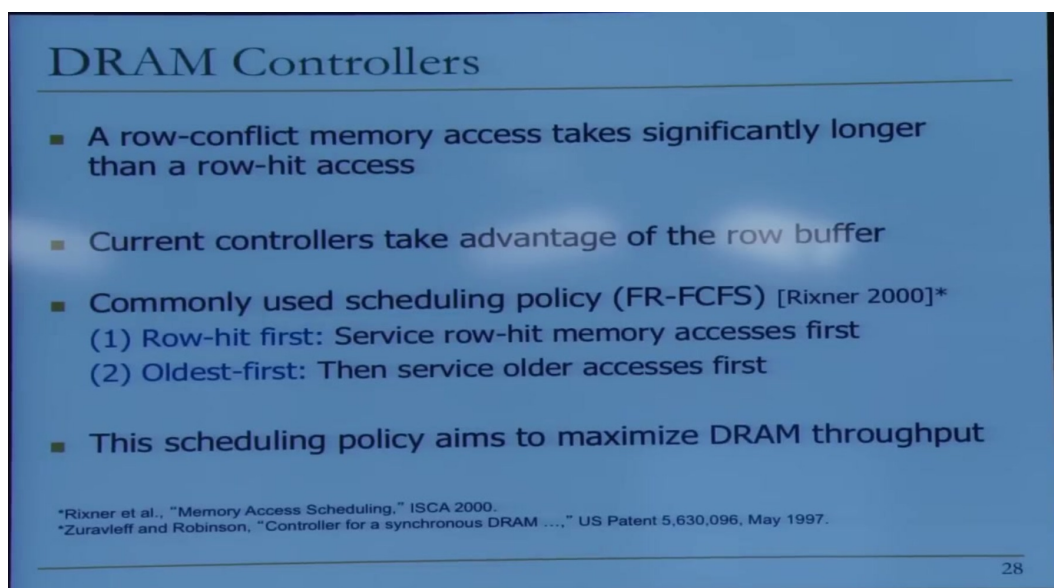


图 19: FR-FCFS 调度优先服务 row buffer hit，再按请求年龄排序；它提升吞吐，也可能牺牲应用间公平性。¹⁹

这个策略有直接的工程理由。若 bank 里 row 0 已经打开，队列里又有多个访问 row 0 的请求，连续服务它们可以避免反复 precharge 和 activate 的开销，吞吐会提高，请求平均延迟也会降低。在单核、单程序时代，这种策略通常很好，因为所有请求都来自同一个应用，row buffer locality 直接转化为该应用自己的性能收益。

多核系统改变了问题的含义。若一个应用有很强 row buffer locality，持续向同一个 open row 发请求；另一个应用访问模式随机，经常落在不同 row 上；row-hit-first 策略会反复选择第一个应用。oldest-first 也不一定带来应用级公平，因为队列条目本身可能被某个高强度应用占满。讲者举了一个直观队列例子：如果 memory controller 有 128 个 buffer entry，其中 127 个来自 MATLAB，只有 1 个来自 GCC，那么“选最老请求”很大概率仍会选中 MATLAB 的请求。

throughput 与 fairness 的张力

throughput 关注单位时间完成多少 DRAM 请求；fairness 关注不同应用相对于独占运行时受到多少伤害。row-hit-first 策略能提高吞吐，原因是它复用 open row；同一策略也可能压低公平性，原因是它偏向具有高 row locality 和高请求强度的应用。两个目标都真实存在，架构设计必须说明优先级和代价归属。

这正是 MATLAB/GCC puzzle 的机制。MATLAB 在该执行中像流式应用：它产生大量、连续、row locality 很高的内存请求，并且许多访问都是 cache miss。GCC 的请求较少，也更容易被 MATLAB 的请求海量包围。memory controller 为了提高 row hit rate，不断服务 MATLAB 的 open-row 请求，GCC 的少量请求即使早已在队列里，也可能长期得不到服务。

5.5 为什么一个 row 可以连续压住 128 个 cache block

讲者用一个简单算术说明这种压制可以持续多久。直观地说，DRAM 打开一个 row 后，这个 row 里包含很多 cache block；流式访问若快速扫过这一整行，就能连续制造 row hit。若一个 row 是 8 KiB，一个 cache block 是 64 B，那么同一个 open row 里有 128 个 cache block。

$$N_{\text{blocks}} = \frac{\text{row size}}{\text{cache block size}} = \frac{8192 \text{ B}}{64 \text{ B}} = 128$$

¹⁹视频画面时间区间：00:44:52–00:45:59。若图像为裁剪图，时间区间仍对应原视频画面。

- N_{blocks} : 一个 DRAM row 中可容纳的 cache block 数量。
- row size: DRAM row 的大小。
- cache block size: cache 与内存之间传输和管理的基本块大小。
- 8192 B: 8 KiB 换算成 byte 后的 row 大小。
- 64 B: 例子中的 cache block 大小。
- 128: 该例子中同一 row 可提供的 cache block 数量。

这个公式给出的是一个机制放大器，具体倍数会随 row 大小、cache block 大小和访问模式变化。只要 row 足够大、cache block 足够小、流式应用能持续送来请求，controller 就可能在服务一个随机应用请求之前，连续服务同一流式应用的大量 row-buffer-hit 请求。GCC 被 MATLAB 拖慢三倍，背后就有这种“一个 open row 反复被利用”的路径。

128 的教学意义

这里的 128 说明 row-buffer locality 可以把一个很小的调度偏好放大成很长的等待时间。每次选择 row hit 看起来都合理；连续 128 次选择同一应用后，另一个应用看到的就是严重 slowdown。这是共享资源设计中常见的放大效应。

5.6 从拒绝服务到公平 memory controller

讲者进一步指出，这种机制可以被程序利用。一个流式、内存密集、row locality 很强的程序，能让另一个低 row locality 的程序长时间等待；这已经接近 denial of service 的结构。这里的“攻击”不需要破坏硬件，只要构造合适的访问模式，就能利用调度策略的偏好。

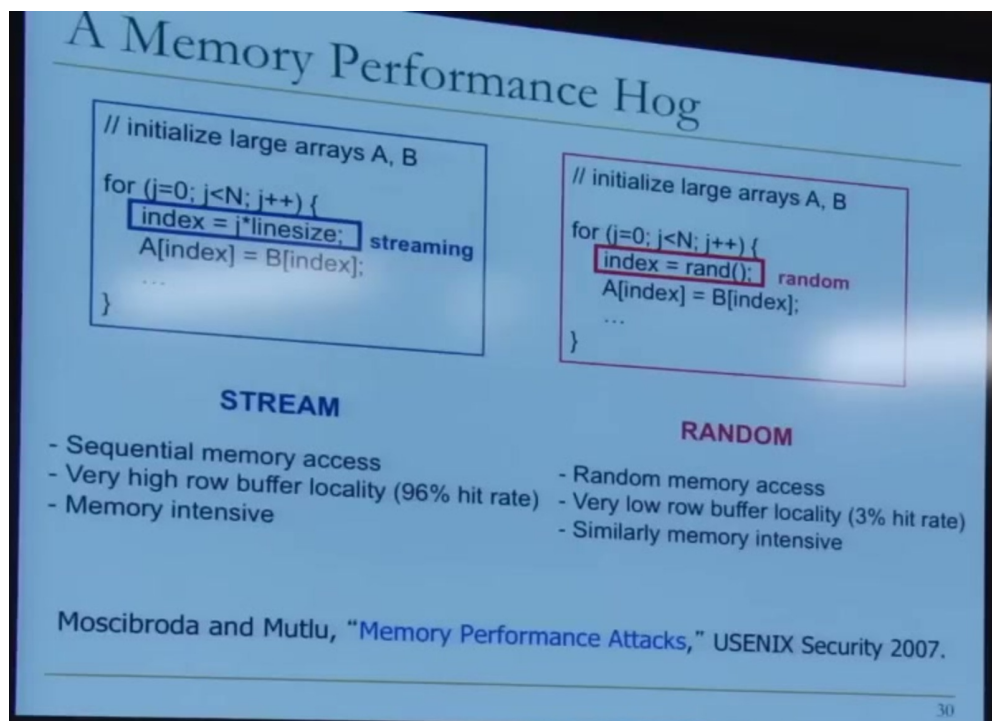


图 20: STREAM 和 RANDOM 都很吃内存，差异在于 STREAM 有很高 row buffer hit 率，RANDOM 的行局部性很低。²⁰

²⁰视频画面时间区间：00:47:34–00:48:32。若图像为裁剪图，时间区间仍对应原视频画面。

图中还有一个值得保留的数量对比: STREAM 的 row-buffer hit rate 约为 96%, RANDOM 约为 3%。二者都可能很 memory-intensive, 真正拉开调度命运的是 row locality。对 memory controller 来说, STREAM 连续送来的请求像不断命中同一页纸, 服务起来局部高效; RANDOM 经常换 row, 单个请求更容易排在一串 row-buffer-hit 请求之后。

解法可以放在多个层次, 但每个层次的可见信息不同。

- Programmer: 程序员通常不知道自己的程序会和谁同跑, 也不知道共享 DRAM 队列中的实时竞争状态, 因此很难做通用修复。
- Compiler: 编译器看到单个程序的代码与优化机会, 对其他进程的访问流通常缺少运行时信息。
- Operating system: 系统软件可以通过调度、页分配、channel/bank 映射等方式影响干扰, 但常规 CPU priority 难以直接控制 memory controller 的内部服务顺序。
- Memory controller: 它能看到来自多个 core 的 request streams 与 bank 状态, 因此是本案例中最直接的控制点。
- DRAM design: 若 DRAM 里有更强的内部控制逻辑, 理论上也可能参与缓解; 常规 DRAM 芯片对应用身份和系统策略的可见性较低。
- Circuits: 电路层负责实现时序、电荷感知和物理访问, 抽象层级太低, 难以独立表达“应用公平性”。

一种直接方向是设计 fair memory controller: 它评估不同应用受到的 slowdown 或服务失衡, 在调度时纳入公平性, 把 row hit rate 从唯一目标调整为多个目标之一。另一个方向是系统软件做映射: 如果系统有多个 channel 或 bank, 可以把一个应用的数据映射到一个 channel, 把另一个应用的数据映射到另一个 channel, 减少请求流在同一共享资源上的碰撞。还有一个局部方向是缩小 controller buffer, 并对进入 buffer 的请求做公平分配; 代价是较小 buffer 可能减少 memory-level parallelism, 降低系统利用 DRAM 并行性的能力。

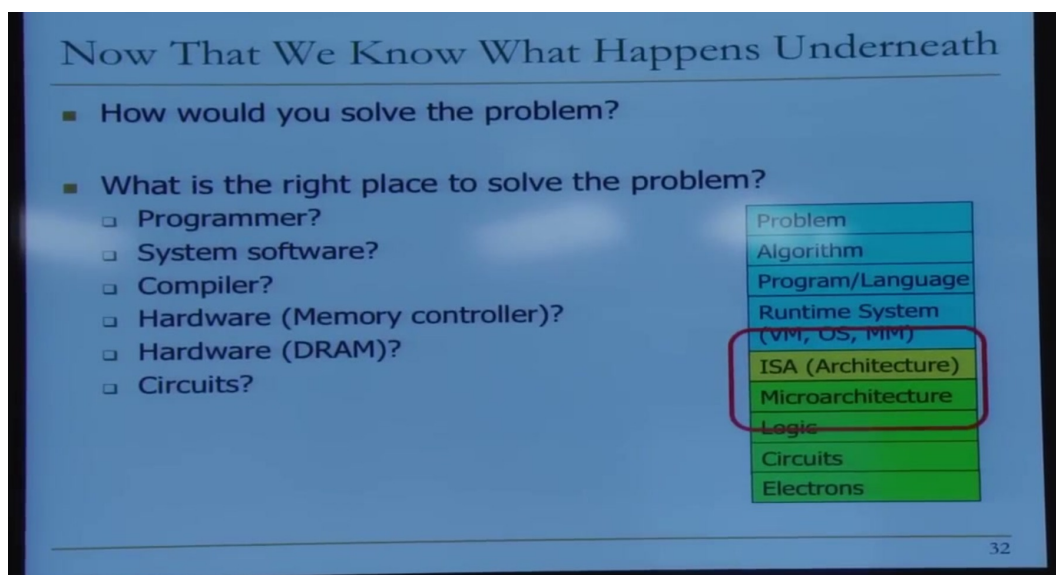


图 21: 内存干扰问题跨越程序、系统软件、编译器、内存控制器、DRAM 和电路层, 解法位置本身就是架构判断。²¹

²¹ 视频画面时间区间: 00:50:49–00:51:39。若图像为裁剪图, 时间区间仍对应原视频画面。

为什么 memory controller 是这个案例的关键可见点

memory controller 同时知道三类信息：第一，哪些请求来自哪些 core 或应用；第二，当前 bank 打开了哪一行；第三，请求队列里谁等待了多久。公平调度需要把这些信息合在一起。上层软件看不到完整 bank 状态，DRAM 芯片通常看不到应用级目标，controller 正好位于两者之间。

解法位置决定能处理的信息

把问题放到更高层，程序员和 OS 更容易表达策略目标，例如服务质量、应用优先级或隔离需求；把问题放到更低层，memory controller 和 DRAM 更接近真实冲突点。好方案往往需要跨层协作：上层给出意图，下层根据 bank、row buffer 和队列状态执行。

5.7 本章小结

本节用 00:35:59–00:56:53 的 MATLAB/GCC 案例展示了多核系统中的共享资源干扰。第一，slowdown 应以应用单独运行作为基准， $\text{slowdown}_i = T_i^{\text{shared}}/T_i^{\text{alone}}$ 能直接表达共享执行带来的伤害。第二，OS priority 在本案例中没有改变结果，因为两个应用已经分别运行在两个 core 上，真正排队的位置在 memory controller。第三，DRAM bank 通过 row、column 和 row buffer 组织访问；row buffer hit 很快，row conflict 需要 precharge 和 activate，延迟更高。第四，FR-FCFS 风格调度优先 row buffer hit，能提升吞吐，也会偏向高 row locality、高请求强度的应用。第五， $8\text{ KiB}/64\text{ B} = 128$ 说明一个 open row 可以连续贡献 128 个 cache block，流式应用因此可能让随机应用长期等待。第六，解法需要放在能看见问题的位置：fair memory controller 是本段课的直接答案，系统软件的 channel/bank 映射、buffer entry 分配和跨层协作也是可行方向。

6 案例二：DRAM Refresh、扩展压力与 Retention-Aware Design

6.1 从共享资源干扰走向器件物理

上一节的 MATLAB/GCC 案例已经给出一个核心训练：当系统出现异常行为时，架构师需要越过单一抽象层，查看隐藏在接口下方的真实机制。DRAM refresh 是同一思想的第二个案例，只是问题从多核性能干扰转向了器件物理、能耗、可扩展性和可靠性。讲者在 00:56:53–01:23:53 这一段强调，今天看似底层的 DRAM 单元泄漏，会在未来大容量系统里变成明显的系统级开销。

DRAM 的吸引力来自密度。一个 DRAM cell 用电容保存电荷，用访问晶体管连接 wordline 和 bitline；许多 cell 组成 row，许多 row 组成 bank 和芯片。电容里有电荷时表示一种逻辑值，电荷较低时表示另一种逻辑值，具体的 0/1 对应取决于编码和感测电路。这个模型像一个非常小的水桶：能装水，成本低，排列密集；同时桶壁会漏水，水位迟早下降。

泄漏让 DRAM 获得了一个额外动词：refresh。memory controller 需要周期性激活每一行，把电荷恢复到可读状态。这个动作本身占用 DRAM 接口和 bank/rank 资源，也消耗能量。应用程序在等待一次普通访存时，可能正好撞上 refresh；它看到的是延迟增加和服务不确定性，底层原因则是电荷保存的物理限制。

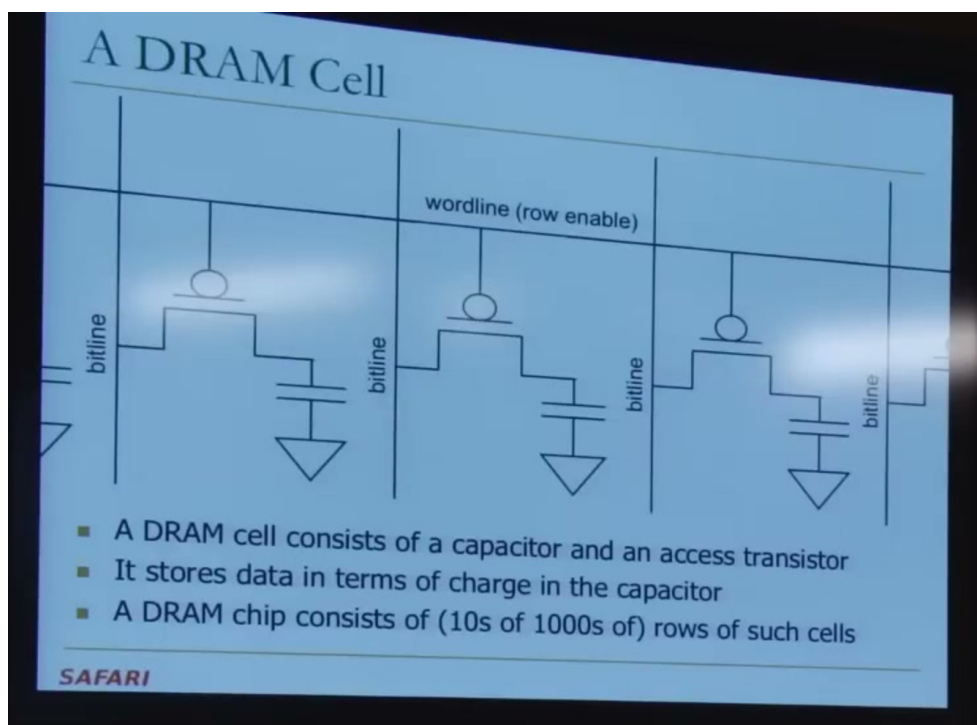


图 22: DRAM Cell 以电容电荷表示数据，电荷会随时间泄漏，因此需要周期性 Refresh。²²

DRAM refresh 是器件物理向系统行为上传的典型例子

DRAM cell 通过电容电荷保存数据，电荷会沿着泄漏路径逐渐消失。refresh 的任务是在数据失真前恢复电荷。它看起来像内存芯片内部维护，实际影响包括性能、能耗、可预测性和扩展能力；因此架构师需要同时理解 cell、row、memory controller 和操作系统可见性。

为什么 **cache** 通常无需这一类 **refresh**

课堂里有学生问 cache 是否也由 DRAM 构成。讲者的回答是：今天的 cache 通常由 SRAM 构成。SRAM cell 使用更复杂的晶体管结构维持状态，面积更大、密度更低，访问速度快，也无需像 DRAM 电容那样按固定窗口恢复电荷。这个差异解释了为什么片上 cache 更快也更贵，而主存 DRAM 更密集也要承担 refresh。

6.2 64 ms refresh window 与刷新次数

先把三个容易混淆的量分开。refresh window 指系统要求覆盖所有行的一轮刷新窗口；refresh interval 指某一行或某类行被安排刷新的间隔；retention time 指电荷在无 refresh 时仍可安全保存的时间。讲者使用的典型 refresh window 是约 64 ms。这个数值是课程中采用的常见设计窗口，具体芯片、温度、电压和工作模式会带来不同边界。只要系统采用统一 refresh，窗口内要覆盖全部 row；容量越大、row 越多，刷新工作总量越大。

这个关系可以写成下面的刷新窗口与刷新次数公式。第一项把容量换成 row 数，第二项说明一个 refresh window 内要刷新多少 row，第三项给出单位时间刷新速率。

$$N_{\text{rows}} = \frac{C_{\text{DRAM}}}{S_{\text{row}}}, \quad N_{\text{win}} = N_{\text{rows}}, \quad R_{\text{uniform}} = \frac{N_{\text{rows}}}{t_{\text{REFW}}}, \quad t_{\text{REFW}} \approx 64 \text{ ms}$$

²²视频画面时间区间：00:58:04–01:00:54。若图像为裁剪图，时间区间仍对应原视频画面。

- N_{rows} : DRAM 系统或芯片中的 row 总数。
- C_{DRAM} : 被考虑的 DRAM 容量。
- S_{row} : 一个 DRAM row 的容量。
- N_{win} : 一个 refresh window 内需要完成的 row refresh 数量。
- R_{uniform} : 统一 refresh 策略下的 row refresh 速率。
- t_{REFW} : refresh window, 讲者使用的典型值约为 64 ms。
- 64 ms: 课程中用于分析的常见 DRAM refresh window 数值。

课堂里的思考题把这个公式放大到极端规模。若按二进制量级把 1 EB 视作 2^{60} bytes, 并假设一个 row 是 8 KiB = 2^{13} bytes, 那么 row 数达到 2^{47} 。在统一 refresh 下, 系统每 64 ms 就要覆盖这 2^{47} 行。这个例子故意夸张, 却揭示了真实趋势: 主存容量持续增长时, refresh 不再只是芯片内部的小维护动作, 它会成为电源、吞吐和时延预算里的硬开销。

不要把 64 ms 当成脱离环境的自然常数

refresh window 来自器件能力、温度、电压、标准规范和系统设计余量的组合。温度升高、工艺缩小、弱 cell 增多、目标可靠性变化, 都会改变安全窗口和设计选择。课堂使用 64 ms 是为了建立统一 refresh 的基线, 后面的 retention-aware refresh 会利用 row 保持能力差异来降低开销。

6.3 刷新开销怎样限制 DRAM scaling

refresh 的第一类代价是能耗。每次 refresh 都要重新激活和恢复电荷, 消耗芯片能量; 在讲者的能耗图里, 当前量级下 refresh 已经可能占据约 15% 的 DRAM energy, 未来大容量芯片上甚至可能接近一半。图像证据把 energy overhead 单独量化: 在 refresh energy 图中, 4 Gb 芯片约有 15% 的 DRAM energy 用于 refresh, 外推到 64 Gb 时约为 47%。第二类代价是性能。正在 refresh 的 bank 或 rank 通过相同接口执行内部维护, 普通读写无法同时使用同一资源, 吞吐被直接切走。第三类代价是可预测性。应用发出的访存请求可能排在 refresh 后面, 延迟出现额外抖动; 实时系统或服务质量敏感系统会感到更强压力。

讲者给出一个高信号 scaling 图: 横轴是 DRAM chip capacity, 纵轴是芯片花在 refresh 上的时间比例。以 4 GB 芯片为例, 设备可能把约 8% 的时间花在 refresh 上, 相当于这部分吞吐不可用于普通访问。若继续按相同思路扩展到 64 GB 芯片, row 数增加、工艺层面的 refresh 压力也增加, 图中外推显示约 46% 的时间会花在 refresh 上。performance 图的 8% → 46% 与 energy 图的 15% → 47% 互相支撑: refresh 同时吞掉可用时间和能量预算, 且两者都会随容量扩展而快速变成系统级问题。

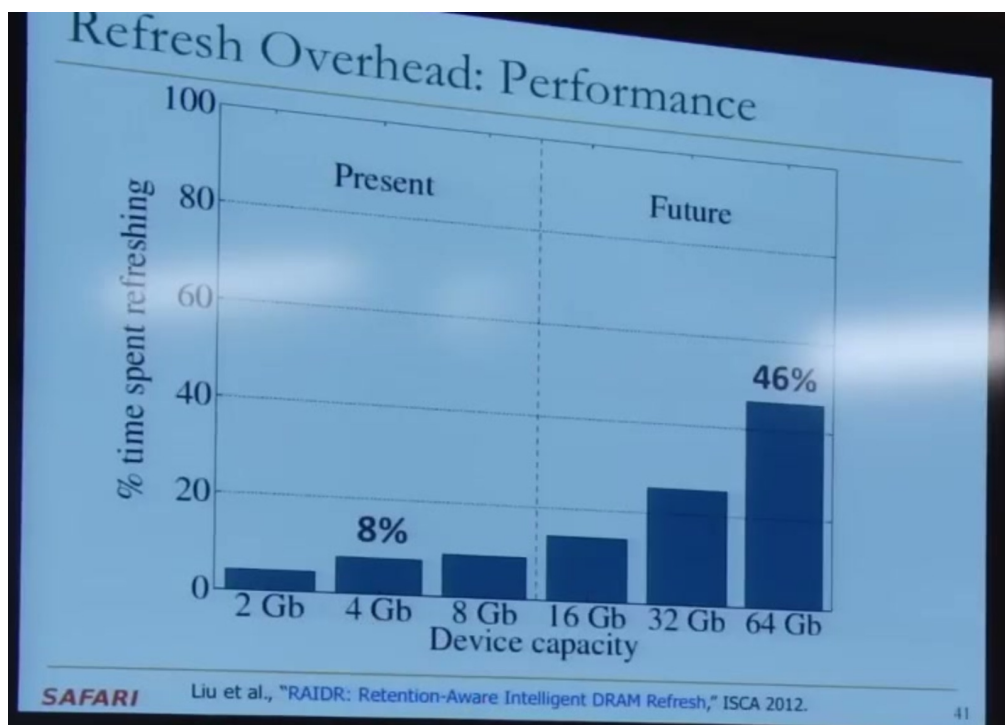


图 23: Refresh overhead 的性能曲线显示：容量增长会让刷新占用越来越多可用时间。²³

学生提出了一个自然替代方向：既然大芯片 refresh 负担重，可否增加更多 DRAM channel 来获得带宽和容量。讲者承认这是一条真实 tradeoff 线，同时指出 channel 很贵，因为 channel 需要芯片引脚、封装资源、板级布线和控制逻辑；课堂中提到许多系统每个 channel 大约需要 144 个 pins。pin 是稀缺资源，靠无限增加 channel 来避开 refresh scaling 压力并不可持续。

scaling 问题的本质

DRAM 容量增长会增加 row 数，工艺缩小会增加弱 cell 和保守 refresh 的压力。统一 refresh 把最弱 row 的要求施加给全部 row，于是容量越大，浪费越明显。架构师需要寻找更精细的可见性和控制点，使 refresh 工作量接近真实需要。

6.4 Retention variation: 为什么统一刷新会浪费

解决 refresh 开销的关键问题是：所有 row 是否都需要每 64 ms refresh 一次？课堂先给出一个上层可见性的想法：若操作系统知道某些物理内存区域没有被分配，memory controller 又能接收这个信息，那么这些区域可以少做 refresh，甚至在安全条件下暂停 refresh。今天常见系统中，这种 OS 到 memory controller 的协调机制很有限；没有分配信息的 controller 会继续刷新整片 DRAM。

更重要的方向来自芯片内部：不同 row 的 retention time 差异很大。retention time 指一个 cell 或 row 在无 refresh 情况下仍能安全保存数据的时间。讲者展示的分布里，绝大多数 row 能保持超过 256 ms，只有少数 row 需要 64 ms 或 128 ms 级别的 refresh。制造过程中的 process variation 是根本原因：电容面积、晶体管特性、泄漏路径、局部缺陷都存在差异；有些 cell 更强，有些 cell 更弱。

²³视频画面时间区间：01:01:36–01:05:45。若图像为裁剪图，时间区间仍对应原视频画面。

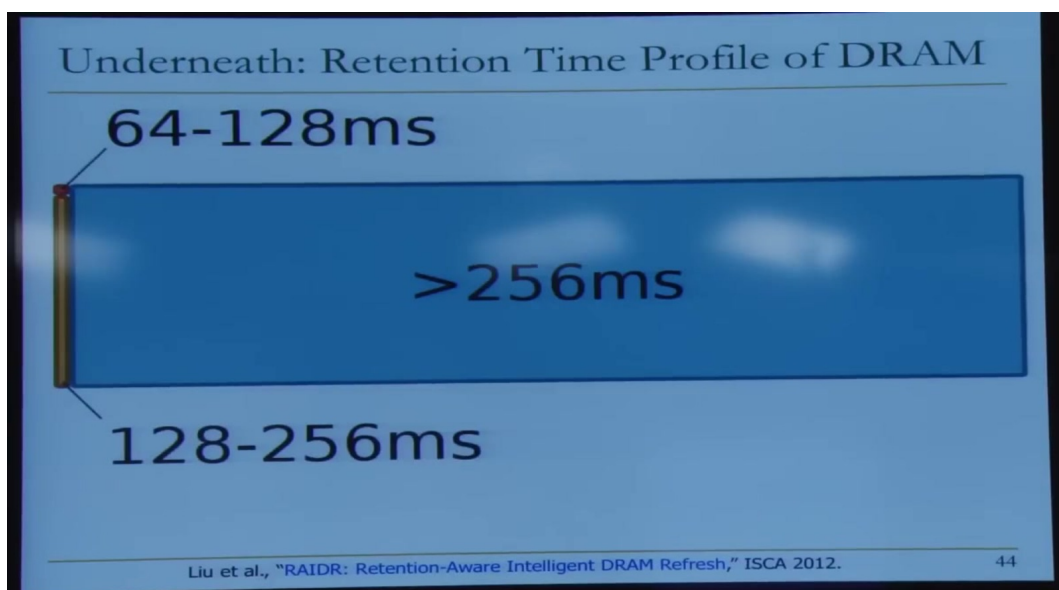


图 24: Retention time 分布说明，多数单元可保留更久，少数弱单元决定了保守统一刷新策略。²⁴

统一 refresh 的保守性可以用“按最低短板定全班作息”来理解。少数弱 row 需要较短窗口，整个芯片就按照短窗口运行。制造商会筛掉或修复低于规格的坏 cell，课堂里还提到一些底层 cell 甚至无法保持 32 ms，它们通常通过芯片内部修复机制被隐藏起来。暴露到系统层面的 row 已经通过了最低门槛，可它们之间仍然存在宽阔的 retention 差异。

retention profile 是跨层优化的“地图”

retention profile 记录哪些 row 能保持多久。没有这张地图，memory controller 只能使用统一保守策略；有了这张地图，controller 可以把 row 分成不同 retention bins，例如 64 ms、128 ms、256 ms 等。地图越细，潜在节省越大，同时 profiling、存储、验证和更新成本也会上升。

6.5 Retention-aware refresh: 按 row 的真实需要调度

retention-aware refresh 的基本机制很直接：弱 row 更频繁 refresh，强 row 更少 refresh。若大多数 row 可以保持 256 ms，少量 row 需要 64 ms，那么 controller 可以把主要 refresh 工作从 64 ms 级别放宽到 256 ms 级别，同时单独照顾弱 row。课堂中讲者用这个想法说明，理想情况下 refresh 数量可以接近减少 75%，因为 256 ms 是 64 ms 的四倍。

²⁴视频画面时间区间：01:07:53–01:11:51。若图像为裁剪图，时间区间仍对应原视频画面。

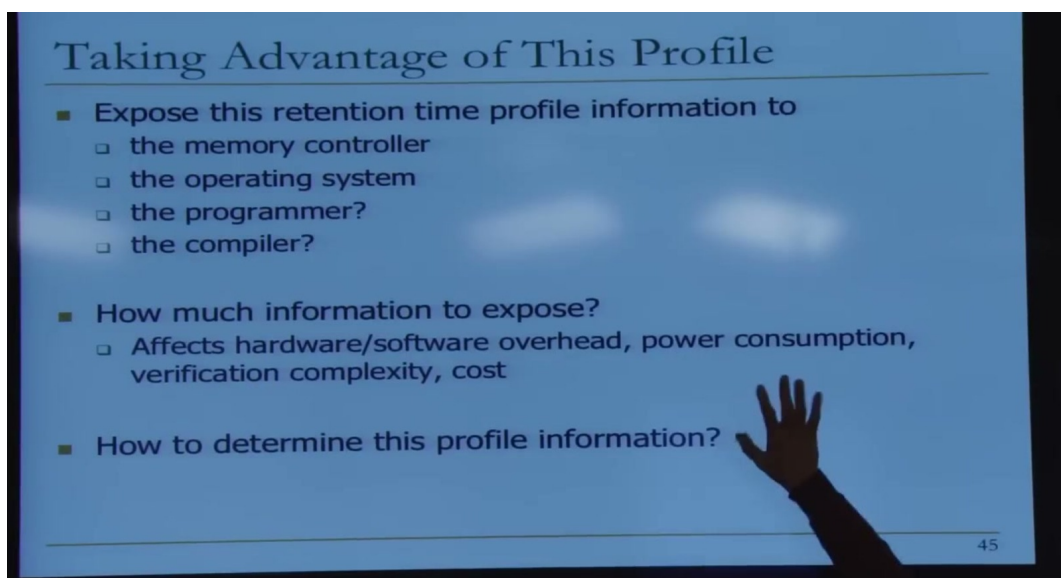


图 25: 按行选择 Refresh interval 的想法: 弱行更频繁刷新, 强行延长间隔, 从而减少无效刷新。²⁵

先用平实语言说 row-specific refresh 的安全条件: 为每一行 r 选择一个 refresh interval T_r , 这个间隔要小于或等于该行测得的 retention time。这样, 在下一次 refresh 到来前, 行里的数据仍然处在可可靠感测的范围内。

$$T_r \leq t_{\text{ret}}(r)$$

- r : 被调度 refresh 的 DRAM row。
- T_r : 系统为 row r 选择的 refresh interval。
- $t_{\text{ret}}(r)$: row r 的 retention time, 即它在无 refresh 情况下可安全保存数据的最长时间。
- $\leq$: 安全约束, 表示 refresh 间隔需要落在该 row 的保持能力以内。

这个公式只写出了安全边界, 真实系统还要解决三个工程问题。第一, 怎样 profile。讲者强调这是最困难的部分: 系统必须知道哪些 row 需要什么 refresh interval, 而且测量要覆盖温度、电压、老化、数据模式和运行环境变化。第二, 怎样 store。逐 cell 暴露 retention time 会产生巨大硬件、软件、功耗和验证成本; 逐 row 存储也可能很贵, 需要压缩表示。第三, 谁使用。memory controller 是自然控制点, 因为它能发起 refresh; 操作系统也可能利用信息, 把短生命周期数据放在 retention 较短的区域, 或者避免分配最弱区域。

²⁵视频画面时间区间: 01:12:10–01:14:41。若图像为裁剪图, 时间区间仍对应原视频画面。

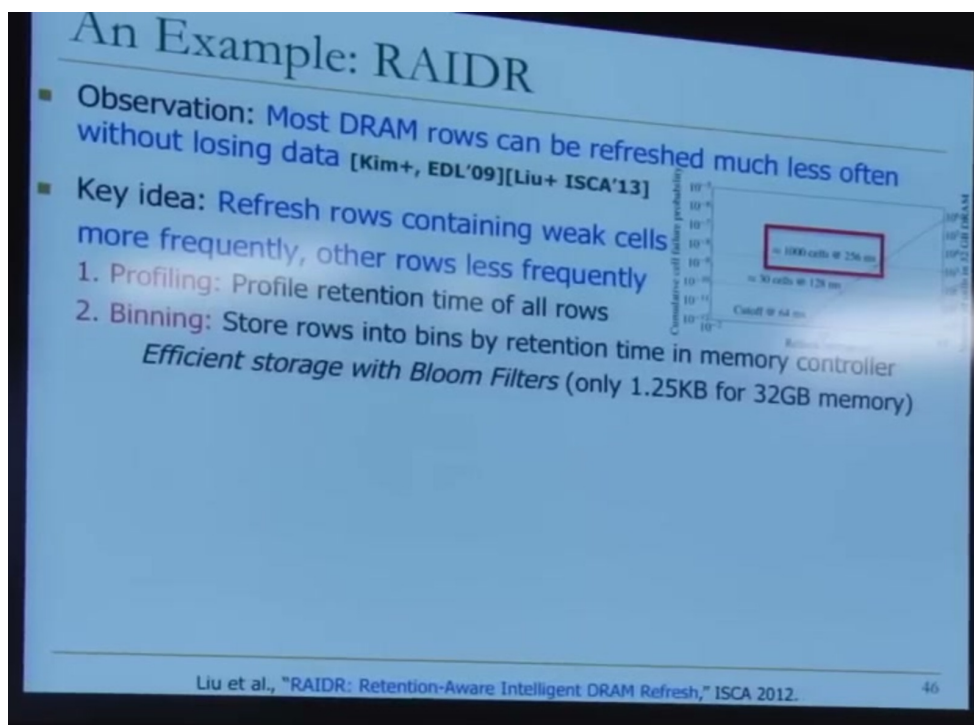


图 26: Retention-aware refresh 需要先测量保留时间，再把行分类信息压缩到控制器可用的元数据中。²⁶

Samsung 数据给了这条路线以实证直觉。课堂提到的外推结果是：在一个 32 GB DRAM system 中，只有约 30 个 cell 需要比 128 ms 更频繁的 refresh；为了这些极少数弱点，统一策略却让全部 row 每 64 ms refresh 一次。这里的架构问题很尖锐：少数弱 cell 的保护需要真实存在，全局保守策略的浪费也真实存在，系统需要用更精细的 metadata 把二者分开。

profiling 和 metadata 是 retention-aware refresh 的硬成本

retention-aware refresh 的正确性依赖 profile 准确性。若 profile 漏掉弱 row，系统可能在 refresh 前读到错误数据。若 metadata 过细，存储和验证成本会吞掉节省。若 profile 随温度、老化或数据模式漂移，旧 profile 会变成风险源。工程设计必须说明测量频率、更新路径、冗余校验、责任边界以及失败后的保守回退。

6.6 Bloom filter（布隆过滤器）：用紧凑 metadata 表示弱 row 集合

为了把 row 按 retention time 分 bin，memory controller 需要知道某个 row 是否属于“需要较短 refresh interval”的集合。直接方法是每个 row 存一个 bit：该 row 是否属于集合。对于 32 GB 内存和 8 KiB row，row 数量约为 $2^{35}/2^{13} = 2^{22}$ ，一 bit per row 已经是百万级 bit。若有多个 bins，成本继续增加。

Bloom filter 提供了一种紧凑表示。它用一个 m -bit bit vector 和 k 个 hash functions 表示集合。插入一个 row address 时，对它计算 k 个 hash 位置，并把这些 bit 设为 1。查询一个 row address 时，也计算同样的 k 个位置；只有这些位置全部为 1，查询才返回“可能在集合中”。

先用直觉说 Bloom filter query 公式：一个地址 x 被 k 个 hash functions 映射到 bit vector B 的多个位置；查询结果是这些位置上 bit 的逻辑与。只要有一个位置为 0，就可以确定 x 没被插入；若全部为 1，系统把它当作集合成员处理。

²⁶视频画面时间区间：01:14:44–01:16:05。若图像为裁剪图，时间区间仍对应原视频画面。

$$\text{Query}(x) = \bigwedge_{j=1}^k B[h_j(x)]$$

- x : 被查询的 row address, 或更一般的集合元素。
- $\text{Query}(x)$: Bloom filter 对 x 的 membership query 结果。
- $\bigwedge$: 逻辑与运算, 要求所有参与项都为真。
- j : hash function 的编号。
- k : Bloom filter 使用的 hash function 数量。
- B : Bloom filter 的 bit vector。
- $h_j(x)$: 第 j 个 hash function 把 x 映射到的 bit 位置。
- $B[h_j(x)]$: bit vector B 在位置 $h_j(x)$ 上的 bit 值。

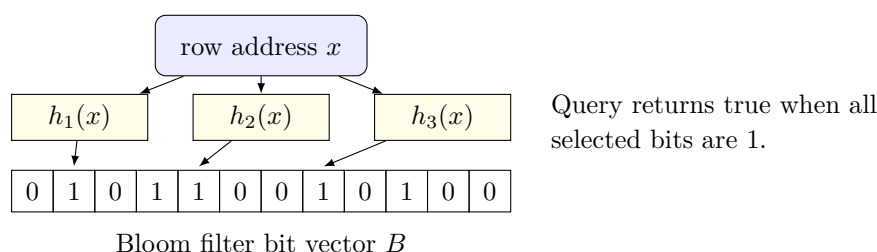


图 27: Bloom filter 用多个哈希函数和位向量表达集合成员关系, 适合压缩保存需要更频繁刷新的行集合。²⁷

Bloom filter 的关键性质非常适合这个 refresh 场景: false positive 可以接受, false negative 必须避免。false positive 指某个 row 实际无需短 interval, Bloom filter 却返回“需要短 interval”。后果只是多 refresh 这行, 浪费一点性能和能量, 数据安全仍然保住。false negative 指某个真正需要短 interval 的弱 row 被查询成“不需要”。后果是 controller 可能把它放到较长 interval, 数据可能在 refresh 前丢失。标准 Bloom filter 在插入和查询实现正确、bit vector 无损坏、hash functions 一致时不会对已插入元素产生 false negative; 系统级设计仍要防范 profile 错误、metadata 损坏、初始化遗漏和运行时漂移。

Bloom filter 的直观类比

可以把 Bloom filter 想成一个压缩签到表。每个 row address 来签到时, 会在若干个位置盖章; 查询时检查这些章是否都在。两个不同 row 可能盖到相同位置, 于是某个没有真正签到的 row 看起来像签到过, 这就是 false positive。已经签到的 row 对应位置已经被盖章, 正常查询会返回存在, 因此没有理想语义下的 false negative。

refresh 场景里两类错误的严重性不同

false positive 会把强 row 当弱 row 处理, 增加 refresh 开销; false negative 会把弱 row 当强 row 处理, 威胁数据正确性。这个差异决定了 Bloom filter 在这里可用: 它把存储成本压低, 同时把概率性错误限制在“多做 refresh”这一侧。工程实现还需要 metadata ECC、保守回退和 profile 复核来守住 false negative 风险。

²⁷视频画面时间区间: 01:16:01–01:20:18。若图像为裁剪图, 时间区间仍对应原视频画面。

6.7 收益、边界与跨层含义

课堂给出的结果显示, retention-aware refresh 可以显著减少 refresh 数量, 接近 75% 的直觉上限; 由于仍有少数 row 需要短 interval, 以及 Bloom filter false positives 会带来额外 refresh, 实际节省低于完美四倍。讲者还提到, 在一个 32 GB 系统里, 用约 1.25 KB 的硬件 metadata 就可以存放关键 bits; 与 one bit per row 的 2^{22} bit 量级相比, 这是很大的压缩。结果层面, refresh 减少带来约 20% DRAM power reduction 和明显性能改善; 容量继续增长时, 收益还会增加, 64 GB 级别情形下能耗改善可达更高量级。

若保留结果页的精确数值, RAIDR 的收益更清楚: 在 8-core、32 GB、SPEC/TPC-C/TPC-H 这组实验设置下, 图中报告约 74.6% refresh reduction, 约 16% DRAM dynamic power reduction、约 20% idle power reduction, 并带来约 9% performance improvement。字幕还强调, 容量越大收益越高; 到 64 GB 量级时, energy reduction 可达到约 50%。

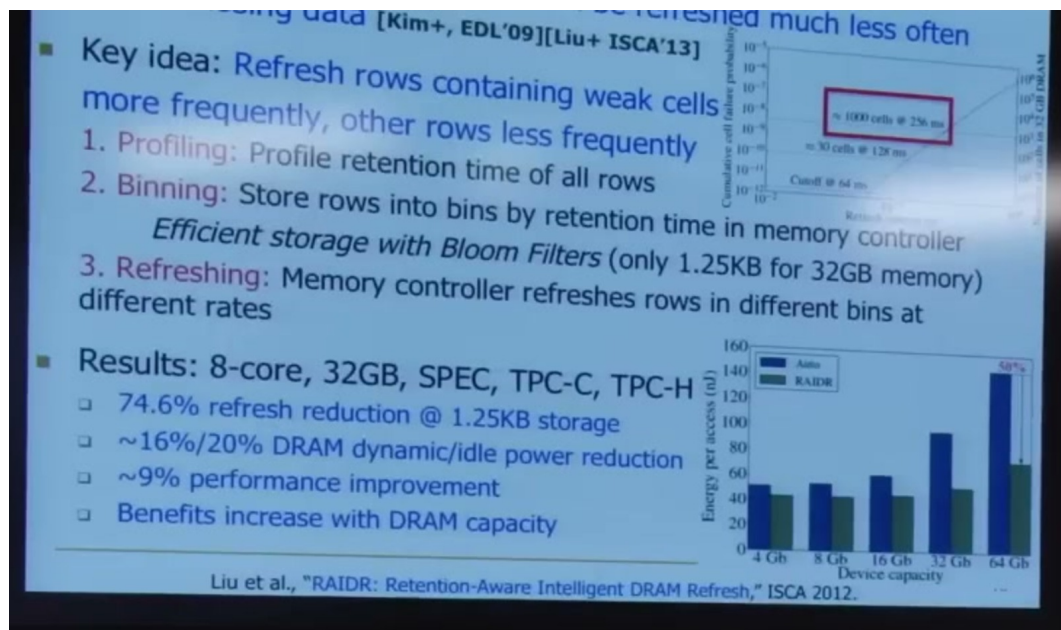


图 28: 存储开销和性能改进结果展示了 retention-aware refresh 的工程收益和元数据代价。²⁸

这段案例的系统含义有三层。第一, 器件信息需要被选择性暴露。逐 cell retention time 太细, 成本不可接受; 逐 row bins 或 Bloom filters 更接近工程可行点。第二, 控制点需要匹配信息。memory controller 能发 refresh, 也能查 row address metadata; 操作系统若获得部分 retention 信息, 还能做物理页分配、短生命周期数据放置、弱区域隔离等策略。第三, 可靠性设计需要保守回退。profile 不确定时, 系统应退回较短 refresh interval; metadata 出错时, 保护数据优先于节省能耗。

本案例的主线

DRAM refresh 从一个 cell 泄漏问题出发, 沿着 row、chip capacity、memory controller metadata、OS 协作一路上传。retention-aware refresh 的价值来自精确区分弱 row 和强 row; Bloom filter 的价值来自用很小 metadata 表示“需要频繁 refresh 的 row 集合”。整体思想是: 把底层物理差异转化为上层可执行策略, 同时把错误类型控制在可接受范围内。

²⁸视频画面时间区间: 01:20:18–01:22:08。若图像为裁剪图, 时间区间仍对应原视频画面。

6.8 本章小结

本节覆盖 00:56:53–01:23:53 的 DRAM refresh 案例。DRAM cell 依靠电容电荷保存数据，电荷会随时间泄漏，因此 memory controller 需要周期性 refresh row。统一策略通常使用约 64ms 的 refresh window；若系统有 N_{rows} 行，则一个 window 内要覆盖全部行，单位时间 refresh 速率为 $N_{\text{rows}}/t_{\text{REFW}}$ 。随着 DRAM capacity 增长，refresh 占用的时间、能量和可预测性成本都会增加，课堂中的外推例子显示 64GB 芯片可能把约 46% 时间用于 refresh。

统一 refresh 的浪费来自 retention variation。process variation 让多数 row 可以保持超过 256ms，少数弱 row 需要 64ms 或 128ms 级别保护。retention-aware refresh 通过 $T_r \leq t_{\text{ret}}(r)$ 的安全约束，为不同 row 选择不同 refresh interval；它需要 profile、metadata、controller 调度和可靠回退。Bloom filter 用 $\text{Query}(x) = \bigwedge_{j=1}^k B[h_j(x)]$ 这样的查询逻辑压缩弱 row 集合表示。false positive 只会造成额外 refresh，false negative 会威胁正确性，因此实现必须保证已插入弱 row 不被漏查，并防范 profile 或 metadata 失效。这个案例把课程前面的跨层思想落到一个硬件现实上：理解底层差异、选择合适暴露粒度、把控制放在看得见问题的位置，系统才有机会在性能、能耗和可靠性之间取得更好的平衡。

7 案例三：RowHammer 与缩放下的可靠性

7.1 从 refresh 走向 disturbance：第三个 DRAM 案例的动机

前一节用 DRAM refresh 说明了一个核心事实：DRAM 的抽象接口给程序员呈现的是“读地址、写地址、数据保持”，底层实际依赖电容电荷、泄漏、定期恢复和 memory controller 调度。RowHammer 把这个事实再往前推进一步。问题已经超出“电荷会自然泄漏，所以系统要 refresh”的范围；即使某个受害行 V 没有被程序直接访问，相邻 row 的高频访问也可能扰动它的电荷，使它在下一次正常 refresh 到来前发生 bit flip。

这正是讲者在 01:23:53–01:38:43 这一段引入 RowHammer 的教学意图：前两个案例已经展示了打破抽象层的价值，RowHammer 进一步说明，缩放到纳米级之后，电路层的耦合效应会穿过 DRAM 接口，变成 architecture、reliability，甚至 security 问题。一个优秀的架构师需要理解程序、OS、memory controller、DRAM bank、wordline、cell 电荷之间的连接关系。

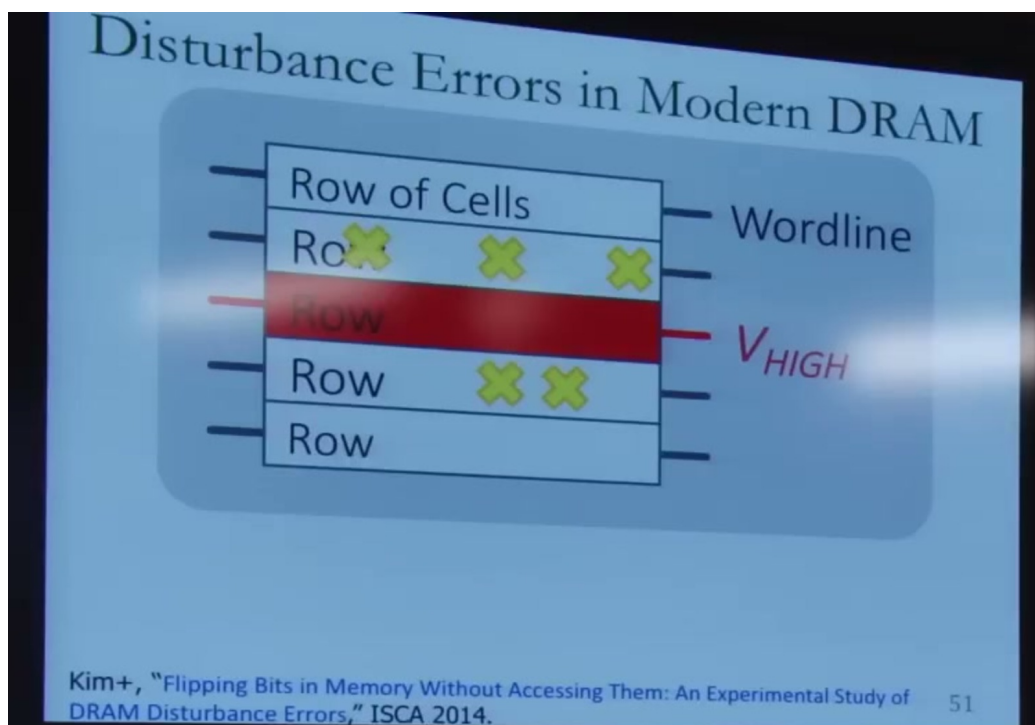


图 29: RowHammer 把 DRAM 缩放带来的扰动错误变成系统可见的可靠性问题。²⁹

RowHammer 的一句话机制

RowHammer 指反复 activate/precharge 同一个 DRAM row，使相邻 row 中部分 cell 的电荷被加速扰动，最终可能在正常 refresh 前降到可靠感测所需范围以下。被反复访问的 row 称为 aggressor row（攻击行），受影响的相邻 row 称为 victim row（受害行）。

disturbance 与 retention 的关系

Retention 问题关注 cell 自己随时间漏电，好比一个水桶静置时慢慢漏水。RowHammer 关注邻近结构受到反复电压切换影响，好比旁边的架子被连续震动后，原本稳定的物品也开始松动。两者都表现为电荷不足，根因和系统可控点有所不同：retention 主要由时间窗口和 cell 强弱决定，RowHammer 还由相邻 row 的访问模式和 activation 频率决定。

7.2 基本模型：攻击行、受害行与电荷阈值

先把 RowHammer 压缩成一个简单模型。设被反复打开和关闭的行是 aggressor row（攻击行） A ，与它相邻并可能受扰动的行是 victim row（受害行） V 。每一次访问 A 通常需要一次 activate，把 A 打开到 row buffer；切换到别的 row 前，bank 需要 precharge，把当前 row 关闭并准备下一次 activate。如果程序或者内存访问序列在同一个 refresh interval 内反复对 A 执行 activate/precharge，activation count 记为 N_{act} 。

受害行 V 中的 cell 用电荷 Q_V 表示数据。感测电路需要电荷保持在足够安全的范围内，这里用 $Q_{\min}$ 表示 disturbance threshold 的直觉边界。课堂材料没有给出可普遍套用的精确触发次数，所以这个模型只表达方向关系： N_{act} 越大， V 被扰动的机会越多；若 Q_V 在下次 refresh 前低于安全感测边界，读出值就可能翻转。

$$N_{\text{act}} \uparrow \implies Q_V(t, N_{\text{act}}) \downarrow, \quad Q_V < Q_{\min} \implies \text{受害行可能发生 bit flip}$$

²⁹ 视频画面时间区间：01:24:24–01:25:42。若图像为裁剪图，时间区间仍对应原视频画面。

- A : aggressor row (攻击行), 即被程序或访问序列反复 activate/precharge 的 DRAM row。
- V : victim row (受害行), 即与 A 相邻并可能因电路耦合而失去电荷的 row。
- N_{act} : 在一个相关时间窗口内对 A 执行的 activation 次数。
- Q_V : 受害行中某些 cell 保留的电荷量。
- Q_{min} : 可靠感测所需的最小电荷边界, 用作直觉阈值; 具体数值依赖芯片、工艺、温度、电压、数据模式和测试条件。
- t : 从上一次 refresh 或写入之后经过的时间。

避免把 RowHammer 写成一个固定次数公式

RowHammer 论文和后续系统会讨论触发次数、刷新窗口、访问速率、地址映射和芯片代际差异。当前课堂段落的本地证据只支持“反复 activate/precharge 足够多次会扰动相邻 row”这一机制性结论。把 N_{act} 写成单一精确门槛会掩盖工艺、温度、ECC、memory-controller 策略和 DRAM 内部修复机制的差异。

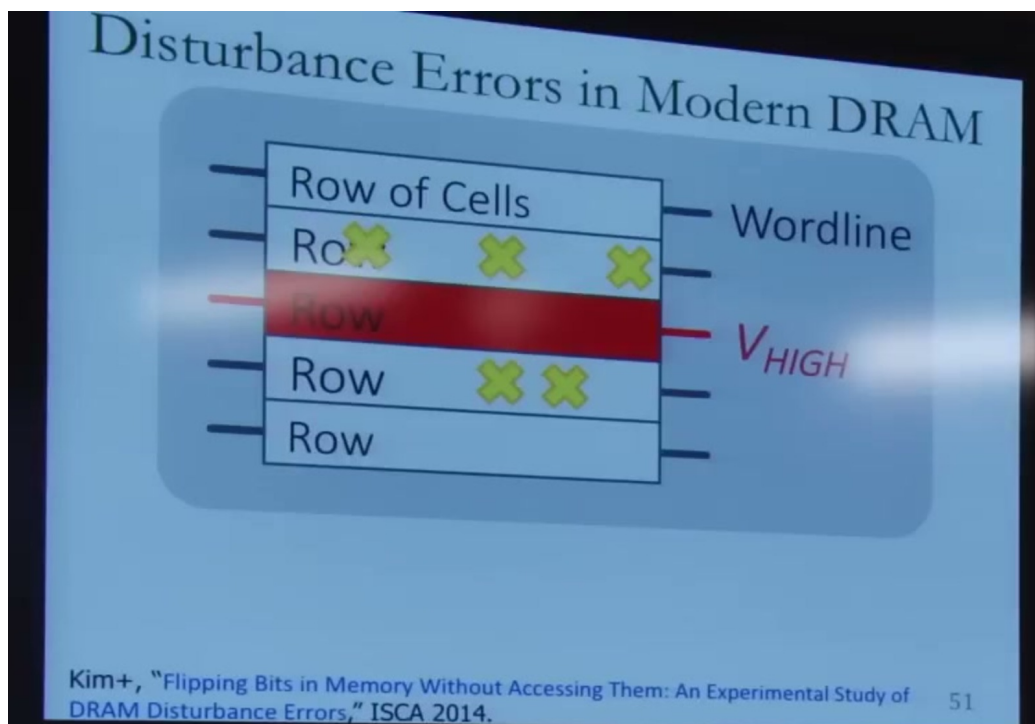


图 30: 攻击行 A 与受害行 V 的相邻关系: 重复激活某一行会影响邻近行中单元保存的电荷。³⁰

7.3 物理机制: 反复 activate/precharge 怎样影响相邻 row

DRAM bank 由大量紧密排列的 row、wordline、bitline 和 cell 组成。访问某个 row 时, memory controller 发出 activate, row 的 wordline 被拉到高电压, cell 连接到 bitline, row buffer 感测并放大数据。切换访问目标时, precharge 让 bank 回到可以打开另一个 row 的状态。这个流程在普通访问中完全正常, 也是前面讨论 row buffer hit 和 row conflict 的基础。

RowHammer 的危险来自频率和邻近性。现代 DRAM 的 feature size 已经进入几十纳米量级, 课堂中提到当时约 25 nm 的尺度, 并且还在继续缩小。相邻 wordline 和 cell 之间的距离越来越小,

³⁰ 视频画面时间区间: 01:25:35–01:26:45。若图像为裁剪图, 时间区间仍对应原视频画面。

电压变化、寄生电容、电磁耦合、漏电路径以及制造变化都更容易产生可见影响。当 A 被反复拉高、关闭、再拉高时，它周围的结构也会受到微小扰动；单次扰动可能很小，高频重复后会在 V 的某些弱 cell 上累计成 accelerated charge leakage。

这解释了一个看似反直觉的现象：程序从未显式访问受害行 V ， V 的数据仍可能改变。DRAM 接口让上层以为 row 之间互不干扰，物理实现中相邻 row 共享硅片空间、金属线、电场环境和噪声余量。缩放让这些隐藏关系更强，抽象层的“独立 row”假设被现实条件削弱。

activate/precharge 重复是 RowHammer 的触发形态

RowHammer 依赖高频 row opening 和 closing。攻击或压力测试程序会构造访问序列，使同一 bank 中的一个或两个 row 被交替访问，从而迫使 DRAM 反复 activate/precharge。只要在 refresh 窗口内积累足够多的 disturbance，邻近受害行就可能在被正常读取时返回错误 bit。

7.4 为什么正常 refresh 可能来不及

Refresh 的作用是周期性恢复 DRAM cell 电荷。若受害行 V 刚刚被 refresh，它的 Q_V 会被补足；若下一次 refresh 又及时到来，普通 retention 泄漏通常被控制在安全范围内。RowHammer 插入了一个新的时间竞争：攻击行 A 在两次 refresh 之间被大量 activate/precharge，受害行的电荷被额外抽走或扰动。于是， Q_V 可能在 refresh window 尚未结束时已经低于 $Q_{\min}$ 。

这可以用一个时间轴理解：上一轮 refresh 把 V 充满；随后程序高频 hammer A ；在下一轮 refresh V 之前，如果 CPU、DMA 或另一个程序读取了 V ，感测电路就可能读到翻转值。下一次 refresh 确实能够补回电荷，可它到达时错误已经可能暴露给系统。换句话说，refresh 能修复周期性自然泄漏，却无法保证覆盖同一窗口内由高频相邻访问造成的突发扰动。

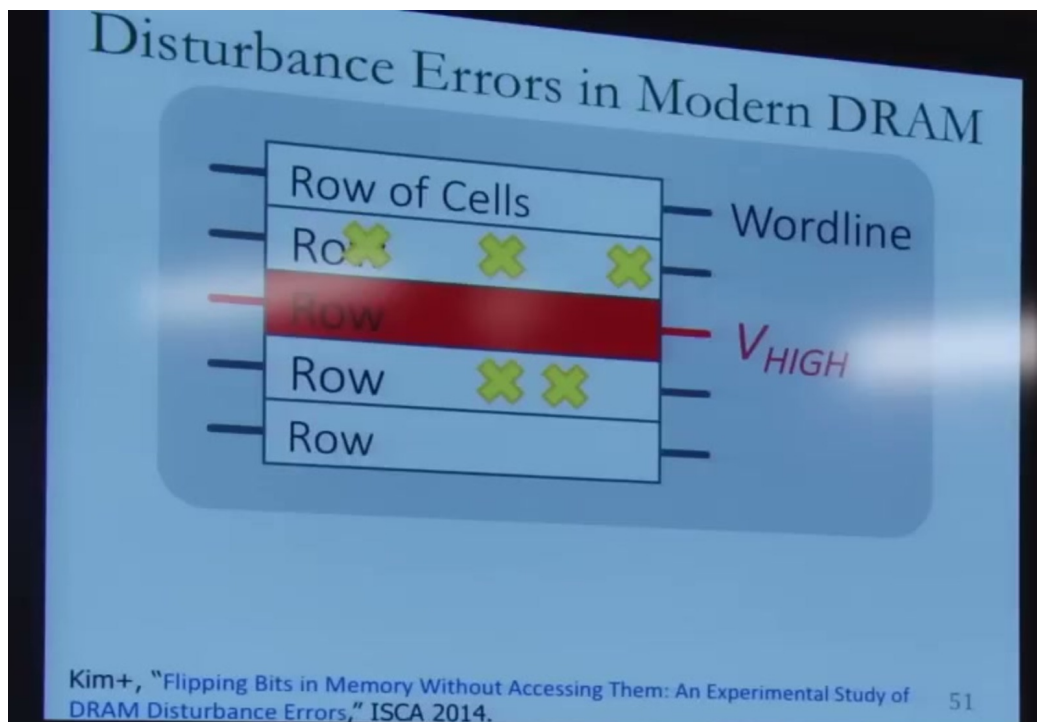


图 31: RowHammer 可在一次刷新周期内部制造电荷扰动，正常 refresh 未必能及时恢复受害行。
31

³¹ 视频画面时间区间：01:24:23–01:26:33。若图像为裁剪图，时间区间仍对应原视频画面。

refresh 对 RowHammer 的保护有时间边界

在 RowHammer 场景中，关键问题是“错误是否在下一次 refresh 之前已经发生并被读取”。只提高常规 refresh 频率可能降低风险，同时带来性能和能耗成本；更有针对性的策略通常会跟踪 hot row，或在检测到高频访问后 refresh 相邻 row。

7.5 商用芯片观察：隐藏电路效应进入真实系统

课堂中讲者强调，RowHammer 已经进入真实芯片测试结果，是商业 DRAM 上可观察的工程问题。研究者测试了来自三家匿名厂商的许多 DRAM modules，并报告超过 80% 的测试 DRAM chips 出现这种 failure mechanism。讲者还指出，可以用真实程序在真实系统上诱发错误；示例程序通过交错访问同一 bank 中的两个 row，提高 activate/precharge 频率，从而在 bank 中造成错误。

这段实证信息有两个特别重要的教学点。第一，问题出现在商业芯片上，说明架构师面对的是实际部署中的可靠性风险。第二，错误行为随制造年份变化：课堂中提到 2010 年以前的 modules 没有观察到这类错误，2012 和 2013 年附近错误达到高峰，随后厂商开始修复相关问题。这里的时间线应按该 2015 年课程和论文语境理解，不能直接外推成所有当前芯片状态。

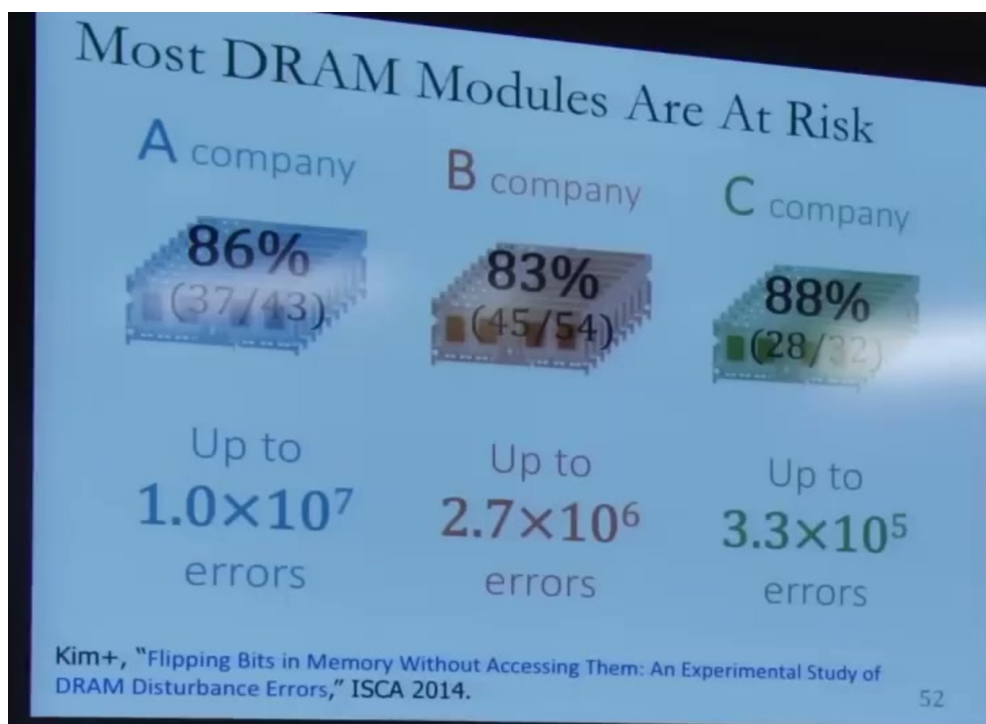


图 32: 不同芯片和年份中的错误数据说明，RowHammer 是真实商业 DRAM 中出现过的缩放可靠性问题。³²

为什么年份趋势很有信息量

如果错误只来自随机软件 bug，通常不会和 DRAM 制造代际呈现强关联。课堂中的年份趋势提示读者，电路缩放、测试覆盖、制造余量和厂商修复策略共同塑造了可靠性。架构师需要把“芯片看起来遵守接口”与“芯片在边界压力下是否保持接口语义”分开验证。

³²视频画面时间区间：01:28:25–01:30:18。若图像为裁剪图，时间区间仍对应原视频画面。

7.6 跨层影响：可靠性问题怎样变成安全和系统问题

RowHammer 的直接表现是 reliability: 某个 bit 从 1 变成 0, 或从 0 变成 1, 系统读到错误数据。更深层的影响来自隔离边界。Victim row 可能属于另一个进程、内核结构、页表、文件缓存或虚拟机。一个用户态程序若能通过自己的访问序列影响相邻物理 row, 就可能把电路 disturbance 转化为跨进程干扰。课堂只简要点出 security 方向, 例如可能导致 system crash; 后续安全课程或论文会展开权限提升、页表 bit flip 等更完整攻击链。

字幕 01:36:57–01:37:14 还给出一个边界条件: 这些 RowHammer 错误发生在 DRAM 中, re-boot 后通常会消失, 因为 DRAM 属于 volatile memory。这个 caveat 没有降低问题严重性; 它说明错误的主要风险窗口在运行时, 包括进程数据、内核结构、页表、缓存内容或正在执行的程序状态。一旦系统重启, 物理存储介质本身通常不会像 persistent memory 那样保留被翻转的位。

这个案例也把 DRAM 放入更广的 scaling 主题。讲者提到硬盘中的 adjacent track interference、flash memory 中的大量原始错误、SRAM 也可能面对类似 disturbance。flash 的经验尤其有启发: 现代 flash controller 用强 ECC、重试、重映射和磨损管理隐藏了大量底层错误。商品 DRAM 长期给上层更强的“可靠主存”印象, RowHammer 说明这种印象需要重新审视。

RowHammer 是跨层可靠性问题

电路层发生耦合和漏电, DRAM command 层表现为高频 activate/precharge, memory controller 层可以观测或抑制 hot-row 模式, OS 层管理物理页和隔离边界, 应用层可能无意触发或有意利用访问模式。任何单层视角都只能看到一部分风险。

7.7 缓解思路：从器件改良到相邻行刷新

讲者把解决方向分成几类, 每一类都有明确成本。第一类是继续改进 device 和 circuit technology, 让 disturbance 更难发生。这条路最直接, 也最依赖制造工艺、cell 设计、布局余量和厂商测试。它的难点同样清楚: 缩放正在强化邻近结构之间的相互影响, 单靠器件层保守设计会牺牲面积、功耗或成本。

第二类是 stronger ECC。ECC 通过冗余 bit 检测并纠正错误, 适合处理一部分随机错误和弱 cell 问题。代价是额外存储、带宽、延迟、功耗和控制逻辑。若 RowHammer 在局部区域产生多 bit 错误, 普通 ECC 可能覆盖不足; 更强 ECC 又会推高系统成本。因此 ECC 是关键防线, 却需要和访问控制、刷新策略、测试筛查一起设计。

第三类是 memory controller 或 DRAM 内部的 hot-row detection。系统跟踪哪些 row 在短时间内被大量 activate; 一旦 N_{act} 接近危险区间, controller 可以延迟后续访问、强制 refresh 相邻 row, 或改变调度策略。这类方案的核心难点是低成本计数。DRAM 有海量 row, 对每个 row 精确计数会消耗大量存储和能量; 近似计数、采样和分层追踪可以降低成本, 也会引入漏检和误报之间的权衡。

第四类是 probabilistic adjacent-row refresh。讲者描述的直觉方案很简单: 每次访问某个 row 时, 用一个有偏硬币决定是否 refresh 相邻 row; 例如平均每十万次或每百万次访问触发一次。由于 RowHammer 需要对同一 row 进行大量重复访问, 低概率刷新会在长重复序列中以较高概率出现, 从而在受害行电荷跌破安全边界前补回电荷。

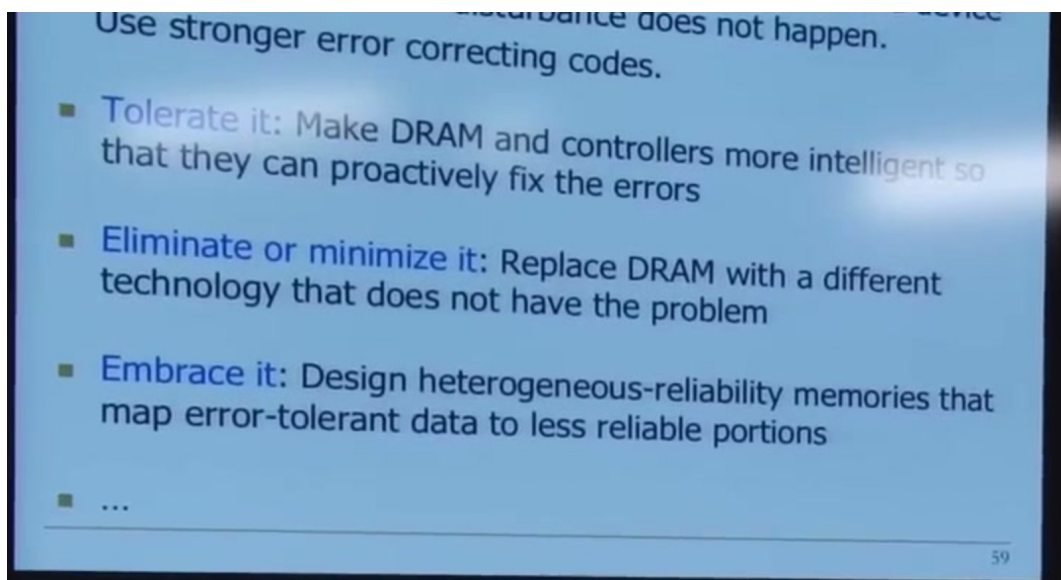


图 33: RowHammer 缓解方向包括检测高频访问行、刷新邻近行、增强 ECC，以及把可靠性责任分布到多层。³³

probabilistic adjacent-row refresh 的直觉

有偏硬币策略像是在高频敲击过程中随机插入“给邻居加固”的动作。单次访问触发概率很低，普通工作负载的额外成本较小；hammer 序列越长，至少触发一次相邻行 refresh 的概率越高。这个思路用概率把昂贵的精确检测转化为低成本风险削减。

缓解方案的风险边界

hot-row detection 需要可靠地识别高频访问，ECC 需要覆盖实际错误形态，probabilistic refresh 需要把触发概率设得足够安全，器件修复需要经受跨温度、跨电压和跨数据模式测试。任何方案若只在平均负载下有效，面对恶意或极端访问序列时仍可能留下缺口。

7.8 更宽的设计空间：可靠内存与可容错内存

讲者还提出一个更激进的架构思路：承认不同数据对错误的敏感度不同，把 memory 设计成不同可靠性等级。高价值数据使用可靠内存，配备更强 ECC、更保守刷新、更严格测试和更高成本；误差容忍数据使用低成本内存，允许偶发 bit flip 由应用语义吸收。视频像素是课堂中的例子：某个像素值从 253 变成 251，视觉影响可能很小；页表、加密密钥、文件系统元数据或科学计算中的关键状态则完全不能这样处理。

这个方向的关键在于“谁知道数据能不能容错”。DRAM 芯片只看到地址和读写命令，通常不了解数据语义；OS 知道物理页归属和权限，编译器或 runtime 可能知道数据结构，应用最了解输出质量需求。若系统要把可靠内存和可容错内存结合起来，就需要新的接口，让语义信息从应用层传到底层，同时让硬件把可靠性等级反馈给软件。这就是跨层架构设计的典型形态。

³³视频画面时间区间：01:32:30–01:34:34。若图像为裁剪图，时间区间仍对应原视频画面。

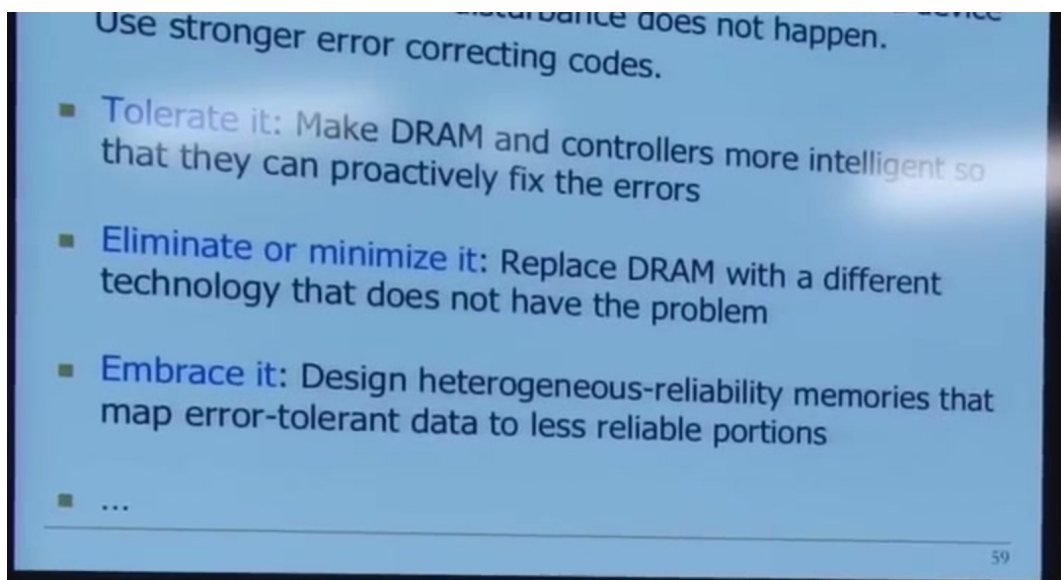


图 34: 扰动错误的解决空间包括更强 ECC、智能控制器、替代技术，以及异构可靠性内存。³⁴

本案例给架构师的设计启示

技术缩放会不断把底层物理效应推到系统接口上。架构师的工具箱包括器件改良、ECC、memory-controller 检测、targeted refresh、概率化防护、OS 物理页管理、应用级容错和新内存技术。真正困难的部分是把正确责任放到正确层，并让这些层通过清晰接口协同。

7.9 本章小结

RowHammer 是本讲第三个 DRAM 案例。它从一个简单操作开始：在一个 refresh interval 内反复 activate/precharge 攻击行 A。由于现代 DRAM row 在纳米尺度上紧密排列，相邻受害行 V 会受到 wordline 和 cell 层面的扰动，某些 cell 的 Q_V 可能在下一次 refresh 之前低于 Q_{min} ，从而产生 bit flip。这里的 N_{act} 是理解风险的关键变量，具体危险阈值取决于芯片、工艺和运行条件。

课堂中的商业芯片观察让这个问题具有工程重量：多个厂商的测试 modules 出现 RowHammer failure mechanism，错误行为还随制造年份变化。它说明可靠性不能只按接口文档推理，还需要测试边界条件、理解电路缩放，并考虑 memory controller、OS、应用和安全隔离之间的耦合。

缓解方案覆盖多个层次：改进 device/circuit，使用更强 ECC，检测 hot-row access pattern，刷新 adjacent rows，采用 probabilistic adjacent-row refresh，或者把内存划分为可靠与可容错区域。所有方案都有成本；RowHammer 的核心课程价值正在于，它迫使架构师把可靠性看成跨层设计问题，避免停留在某个单独组件的局部修补上。

8 课程机制与如何使用这堂课

8.1 把课程网站当作统一入口

讲者在最后一段把注意力从 DRAM scaling 拉回到课程本身：学生要把网站、handouts、syllabus、过往课程网站、邮件通知和 Piazza 看成同一套学习基础设施。网站是资料入口，handouts 给出作业与实验说明，syllabus 给出节奏和规则，过往网站提供可预期的材料形态；Piazza 适合提出和互相回答问题，重要的课程事务仍应通过课程 staff 的正式通信渠道确认。

这里的 logistics 价值是建立一个稳定流程；办公时间和助教信息属于行政入口，服务于资源定位和答疑路径。计算机体系结构的知识分布在讲义、课堂例子、实验、阅读和讨论里，学生若只

³⁴视频画面时间区间：01:35:36–01:36:58。若图像为裁剪图，时间区间仍对应原视频画面。

看其中一项，只能看到单点信息，难以形成机制链。课程网站像一个索引结构，把这些资源连接起来；使用得越早，后续作业和实验越少靠临时补救。

课程网站的教学角色

网站相当于课程的 metadata layer：它告诉学生在哪里找到 handouts、homework、lab、reading、recitation 和历史材料。体系结构课程不断强调“可见信息决定可控行为”，这里也一样；学生能看见完整资源图，才有条件安排时间、预习背景、追踪变动。

课程网站还应与课堂节奏一起理解。字幕 01:43:57–01:46:16 中，讲者说明 attendance 没有硬性要求，参加并互动对学习更有利；课程视频可能在几天后上线，因此学生不能把视频当成立即可用的实时备份。schedule 也是 tentative，材料覆盖范围大体稳定，具体顺序可能调整。讲者用体系结构里的 static scheduling 与 dynamic scheduling 做类比：好的课程运行也会根据实际反馈调整执行计划。

8.2 作业、实验与 recitation 的配合

Homework zero 是学生信息表，讲者强调它是必交项：课程团队需要知道学生是谁，也需要一个行政上的起点。Homework one 随后进入真正的技术内容，包含 refresh 问题、MIPS warm-up、ISA 概念、性能评价，以及论文 critical summary。这里的 MIPS ISA 是软件可见的指令集契约，实验选择 MIPS 的原因是它较适合作为入门 ISA，让学生能把指令、寄存器、状态更新和模拟器行为对应起来。

字幕 01:41:21–01:42:06 给出一条硬规则：Homework zero 承担课程登记与评分前置条件的角色，性质远重于普通热身题。它周五午夜截止，提交入口在课程 handout/Autolab 流程中，讲者还要求附上照片，便于课程团队把学生与沟通记录对应起来。所有 grading 都以收到 Homework zero 为前提；若学生没有提交这份信息表，课程团队无法给出最终成绩。

Lab one 是一个 C-level functional simulator，用 C 语言模拟 MIPS ISA 的一个子集。它和 homework one 形成互相支撑：作业让学生先整理 ISA 和性能评价概念，实验要求学生把这些概念变成可执行的状态转移。recitation 的角色也很明确：它用于增强理解、答疑、讲 MIPS ISA、检查或提交部分实验材料；TA 可以帮助学生定位概念障碍，调试代码仍然属于学生自己的责任。

字幕 01:50:35–01:54:22 给出第一周的精确节奏：Homework zero 本周五截止；Homework one 截止于 Wednesday, January 28th，内容包括 refresh 问题、MIPS warm-up、ISA concepts、performance evaluation 和一篇论文的 half-page critical summary。Lab one 是用 C 写一个 MIPS ISA 子集的 functional simulator，截止于 Friday, January 23rd 的 Friday recitation session 结束时。本周没有 recitation；下周 recitation 会补 MIPS ISA，这正好服务于 Homework one 和 Lab one 的启动。

homework 与 lab 的关系

作业训练“能否解释清楚”，实验训练“能否让机制跑起来”。在体系结构里，这两件事必须合在一起：只会写概念，容易停在抽象层；只会调代码，容易看不见 ISA、microarchitecture、memory controller 等接口含义。Homework one 和 lab one 的协同，正是课程学习方法的早期缩影。

课程规则还有一组需要明写的边界。Homework 可以与同学讨论，目标是逼近真正理解材料；提交物仍然应体现学生自己的推理，并且 late homework 不被接受。Labs 占成绩的重要部分，讲者反复提醒它们会花时间，通常需要个人完成；全学期只有五个 late lab days 可用于实验延期。学术诚信规则更硬：实验代码等材料必须是自己的工作，抄袭会触发 CMU/ECE academic integrity 流程，后果直接上升到 fail course，已经越过单个作业扣分的层级。

评分构成也能解释讲者为什么反复要求 start early。字幕 01:48:21–01:48:54 提到，seven homeworks plus reading summaries 占 14%，此外还有 eight lab assignments、two midterms、a final、

extra credit chances, 以及 staff 对学生整体表现的 subjective evaluation。这里的含义是：课程评价来自持续作业、实验实现、考试反馈和课堂/学习表现的组合，单次冲刺很难覆盖全部风险。

8.3 阅读、写作与时间管理

讲者给出的最硬建议是 start early。课程节奏快，几乎每周都有任务，整体包含八个 labs 和七个 homeworks, labs 占用时间尤其明显。学生若等到 deadline 前才开始，会把体系结构问题误判成单纯实现问题；许多 bug 的真正来源是 ISA 状态、性能指标、共享资源行为或实验规范没有提前读懂。

阅读材料服务于同一目标。Patterson and Patel 第一章可帮助学生回到基础概念；Patterson and Hennessy 第一、二章适合补 MIPS ISA 背景。论文 summary 的要求也很有教学含义：讲者要求学生写半页 critical summary，说明论文思想和方法的 strengths、weaknesses，以及自己学到了什么。阅读的目标是形成判断；摘要只服务于这个判断过程。

时间管理是技术风险控制

在这门课里，拖延会放大三类风险：第一，ISA 和实验环境没有提前熟悉，导致实现时间被低级错误吃掉；第二，论文阅读只剩表面复述，无法形成 strengths 和 weaknesses 的判断；第三，recitation 与 Piazza 的反馈窗口被错过。start early 的含义很朴素：把未知问题提前暴露出来。

8.4 本章小结

本章覆盖 01:38:43–01:54:33 的课程机制。课程网站、handouts、syllabus、过往网站、邮件和 Piazza 构成资源入口；attendance 具有学习价值，视频上线存在延迟，schedule 会根据反馈调整；homework zero 建立课程登记和评分前置条件；homework one 与 lab one 围绕 refresh、MIPS ISA、性能评价和 simulator 实现互相支撑，并带有 January 23rd 与 January 28th 这类近端截止时间；recitation 用于答疑、补充 MIPS ISA、帮助学生理解 labs 和材料；阅读材料包括 Patterson and Patel 第一章，以及 Patterson and Hennessy 第一、二章；论文 critical summary 要求学生评估思想的 strengths、weaknesses 和个人收获。讲者最后给出的学习建议可以压缩成一句话：体系结构课程需要提前准备、持续练习、自己动手、清楚写作，并且把资源、概念和实验放在同一条学习链上。

9 总结与延伸

9.1 一条主线：有原则的跨层设计

这堂导论课从 Fallingwater 开始，看似绕开了计算机，实际是在给体系结构建立第一层直觉。Fallingwater 的价值在于展示设计如何同时回应环境、功能、材料、审美、历史 precedent 和技术约束。计算机体系结构面对的约束换成了 workloads、technology trends、interfaces、cost、power、performance、reliability 和 programmability，设计问题的结构仍然相似：好系统需要在约束中形成清晰原则，并把原则落实到可执行机制。

随后，讲者把这种设计直觉放入 layer stack：problem、algorithm、program、ISA、microarchitecture、logic、circuits、physics/electrons。这个栈像一条翻译链，把人的目标一步步变成物理状态变化。抽象带来生产力，因为上层可以借助稳定接口工作；抽象也制造盲点，因为性能、能耗、可靠性和安全性常常由被隐藏的细节决定。体系结构师的核心能力，就是知道何时遵守接口、何时跨过接口、跨到哪一层、用什么信息做判断。

本讲的压缩命题

计算机体系结构是有原则的跨层设计。它从用户问题和未来 workload 出发, 经过 ISA 与 microarchitecture 等接口, 把底层器件能力转化为上层可用行为; 当共享资源、refresh、RowHammer 这类问题穿透抽象边界时, 架构师必须把被隐藏的信息重新带回设计空间。

9.2 从共享资源到 scaling: 两个案例的共同结构

MATLAB/GCC 与 DRAM memory controller 的案例说明, 共享资源会把局部高效变成系统级 tradeoff。row-buffer-hit-first 调度能提高 DRAM throughput, 因为连续访问 open row 可以节省 activate 和 precharge 的开销; 同一策略在多应用环境中可能造成 unfair slowdown, 因为高 row locality、高请求强度的应用会反复占据 bank 和 controller queue。fairness 在这里指应用相对独占运行时受到的伤害分配, 典型度量是:

$$\text{slowdown}_i = \frac{T_i^{\text{shared}}}{T_i^{\text{alone}}}$$

- i : 被分析的应用。
- T_i^{shared} : 应用 i 与其他应用共享运行时的执行时间。
- T_i^{alone} : 应用 i 独占相关资源时的执行时间。
- slowdown_i : 共享资源对应用 i 造成的相对伤害。

DRAM refresh 与 RowHammer 把同一主线推进到 scaling 和 reliability。DRAM cell 用电荷保存 bit, 电荷会泄漏, 因而需要 refresh; 随着容量增长和工艺缩小, refresh 的时间、能耗和可预测性成本上升。retention-aware refresh 利用 cell 或 row 之间的 retention variation, 把 weak rows 和 strong rows 区分出来, 让 refresh interval 更接近真实需要。RowHammer 进一步说明, 反复 activate aggressor row 会扰动相邻 victim row, 使 victim charge 在 refresh window 内跌破可靠感测阈值。器件物理由此上传为系统可靠性问题, 甚至带来安全含义。

共享资源与 scaling 的共同结构

这两个案例都遵循同一个机制链: 底层事实先被抽象隐藏, 规模、并发或工艺变化放大该事实, 随后上层观察到 slowdown、energy overhead、bit flips 或 reliability risk。有效方案通常需要找到新的可见点: memory controller 看见 request queue 和 bank state; retention profile 暴露 weak rows; hot-row detection 暴露异常 activation pattern。

9.3 课程方法: 用刻意练习打通抽象

讲者的收尾具有超过行政说明的教学价值: 它给出了学习这门课的实际方法。体系结构很难靠旁观获得, 因为它要求学生在多个层次之间来回切换: 读 paper 时要看清问题、假设、机制与证据; 做 homework 时要把概念写成可检验的推理; 做 lab 时要把 ISA、状态转移和性能现象落实到代码; 参加 recitation 和提问时要暴露自己的误解。这里的 deliberate practice 指有目标、有反馈、能修正的反复练习。

Fallingwater 的观察作业、Hamming distance 的形式化定义、DRAM bank 的 row-buffer 机制、retention-aware refresh 的 metadata 设计、RowHammer 的 aggressor/victim 模型, 以及 MIPS simulator 的实验, 都在训练同一件事: 把直觉、形式、机制、实现、评价连起来。学生越早开始, 越容易把“听懂了”转化成“能解释、能计算、能实现、能批判”。

学习路线

本讲建议的学习路线可以写成一个闭环：阅读建立背景，课堂建立机制，作业迫使表达，实验暴露细节，recitation 提供反馈，critical writing 训练判断。每一环都服务于同一个目标：让学生能在抽象层与实现层之间有意识地移动。

9.4 可带走的判断、开放问题与下一步

本讲可以留下五个高密度判断。

- 设计判断：像 Fallingwater 一样，体系结构设计要把目标、环境、历史 precedent 和工程约束合在一起看。
- 层次判断：layer stack 提供生产力，同时需要架构师识别隐藏细节何时变成关键事实。
- 资源判断：共享资源的局部优化需要用 application-level metrics 检查，throughput 和 fairness 都要明确归属。
- scaling 判断：DRAM refresh 和 RowHammer 说明，容量增长、工艺缩小和可靠性风险会改变旧设计的成本结构。
- 学习判断：labs、readings、homeworks、recitations 和 writing 是同一套刻意练习系统，分开使用会削弱学习效果。

后续开放问题也很具体。一个系统应向上暴露多少底层信息，才能让 OS、compiler 或 programmer 做出好决策？memory controller 在追求 row hit、latency、throughput、fairness、energy 和 reliability 时，应如何排序目标？retention profile、Bloom filter metadata、hot-row counters 这类结构如何在低成本、低误报和强可靠性之间取平衡？当未来 memory technology、workload 和 packaging 继续变化时，今天的 ISA、microarchitecture 与 controller 设计会留下哪些长期承诺？

可执行的下一步应从课程安排本身开始：先完成 homework zero，建立课程沟通入口；尽早启动 homework one 和 lab one，把 MIPS ISA、性能评价与 simulator 状态联系起来；按网站阅读 Patterson and Patel 第一章，以及 Patterson and Hennessy 第一、二章；选择一篇与 refresh 或体系结构机制相关的论文写半页 critical summary，明确 strengths、weaknesses 和个人收获；在 recitation 或 Piazza 上把含糊问题尽早问出来。这样使用课程，学生才会把第一讲的核心命题真正落实：体系结构是一门在原则、机制、证据和实现之间反复往返的工程学科。